

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



BÁO CÁO ĐỒ ÁN
MÔN NHẬP MÔN ĐẢM BẢO AN NINH THÔNG TIN
Đề tài: AI HỖ TRỢ QUÉT LỖ HỒNG BẢO MẬT WEB

*Kết hợp Nikto, OWASP ZAP và mô hình ngôn ngữ lớn Claude
để phân tích kết quả quét, đề xuất bản vá và tự đo độ chính xác của AI*

GVHD: [Họ hàm, học vị, họ tên giảng viên]

Nhóm sinh viên thực hiện:

- | | |
|----------------|---------------|
| 1. [Họ và tên] | MSSV: [.....] |
| 2. [Họ và tên] | MSSV: [.....] |
| 3. [Họ và tên] | MSSV: [.....] |
| 4. [Họ và tên] | MSSV: [.....] |
| 5. [Họ và tên] | MSSV: [.....] |

Tp. Hồ Chí Minh, 09/2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

....., ngày.....tháng.....năm 20.....

Người nhận xét
(Ký tên và ghi rõ họ tên)

BẢNG PHÂN CÔNG, ĐÁNH GIÁ THÀNH VIÊN

Bảng 0.1 Bảng phân công công việc và đánh giá thành viên

Họ và tên	MSSV	Phân công	Đánh giá
[Họ và tên]	[MSSV]	Tầng Scanner: chạy Nikto và OWASP ZAP qua Docker; ba profile ZAP (baseline, full, auth); chạy song song; chuẩn hoá kết quả và gom trùng theo fingerprint.	- Tỷ lệ đóng góp: [..]% - Điểm nhóm đánh giá: [..]/10
[Họ và tên]	[MSSV]	Tầng Analyzer: thiết kế prompt và schema đầu ra của AI; chia lô, streaming, cache theo target; nhận diện ngăn xếp công nghệ.	- Tỷ lệ đóng góp: [..]% - Điểm nhóm đánh giá: [..]/10
[Họ và tên]	[MSSV]	Backend và lưu trữ: cơ sở dữ liệu SQLite; API FastAPI; tác vụ quét chạy nền; so sánh hai lần quét.	- Tỷ lệ đóng góp: [..]% - Điểm nhóm đánh giá: [..]/10
[Họ và tên]	[MSSV]	Dashboard và báo cáo: giao diện Jinja2 + HTMX; báo cáo HTML tự chứa; biểu đồ diễn biến số lỗ hổng.	- Tỷ lệ đóng góp: [..]% - Điểm nhóm đánh giá: [..]/10
[Họ và tên]	[MSSV]	Lab và đánh giá: dựng ba target; vòng lặp vá → quét lại; gán nhãn ground truth; đo chỉ số; viết báo cáo.	- Tỷ lệ đóng góp: [..]% - Điểm nhóm đánh giá: [..]/10

LỜI MỞ ĐẦU

Ứng dụng web là nơi doanh nghiệp đặt dữ liệu và nghiệp vụ quan trọng nhất, đồng thời là bề mặt tấn công rộng nhất. Để tìm điểm yếu trước kẻ tấn công, người vận hành thường dùng các công cụ quét lỗ hổng tự động như Nikto và OWASP ZAP. Các công cụ này chạy nhanh và miễn phí, nhưng đầu ra của chúng là hàng trăm dòng cảnh báo thô: trùng lặp, lẫn cảnh báo sai, và hầu như không cho biết cụ thể phải sửa gì. Với người mới học an ninh thông tin, đọc một file kết quả của ZAP gần như không rút ra được hành động nào.

Các mô hình ngôn ngữ lớn (LLM) hiện nay đọc hiểu được văn bản kỹ thuật và viết được mã. Câu hỏi đặt ra là: có thể dùng LLM để biến kết quả quét thô thành danh sách việc cần làm, kèm bản vá dán được ngay hay không, và quan trọng hơn, làm sao biết AI nói đúng? Xuất phát từ câu hỏi đó, nhóm chọn đề tài “AI hỗ trợ quét lỗ hổng bảo mật Web”. Hệ thống nối Nikto và OWASP ZAP với mô hình Claude của Anthropic. Hệ thống không dừng ở việc gọi AI mà còn tự đo độ chính xác của AI bằng nhãn gán thủ công, và chứng minh bản vá bằng vòng lặp áp bản vá rồi quét lại.

Nội dung báo cáo gồm sáu chương:

- Chương 1: Tổng quan đề tài
- Chương 2: Cơ sở lý thuyết và nghiên cứu liên quan
- Chương 3: Phân tích, thiết kế và xây dựng hệ thống
- Chương 4: Môi trường thực nghiệm và phương pháp đánh giá
- Chương 5: Kết quả thực nghiệm và đánh giá
- Chương 6: Kết luận và hướng phát triển

Mọi hoạt động quét trong đề án chỉ thực hiện trên môi trường lab do nhóm tự dựng trên máy cá nhân. Hệ thống được thiết kế để mặc định từ chối quét bất kỳ hệ thống nào nằm ngoài phạm vi đó.

DANH MỤC THUẬT NGỮ

Bảng 0.2 Bảng danh mục thuật ngữ

Thuật ngữ	Giải thích
Lỗ hổng bảo mật (Vulnerability)	Điểm yếu trong phần mềm hoặc cấu hình mà kẻ tấn công có thể lợi dụng
Quét lỗ hổng (Vulnerability scanning)	Dùng công cụ tự động gửi yêu cầu tới hệ thống và phân tích phản hồi để phát hiện điểm yếu đã biết
Phát hiện (Finding)	Một cảnh báo do scanner sinh ra; sau khi gom trùng, mỗi phát hiện ứng với một loại vấn đề
Quét thụ động (Passive scan)	Chỉ quan sát phản hồi bình thường của ứng dụng (header, cookie, nội dung), không gửi payload tấn công
Quét chủ động (Active scan)	Gửi payload tấn công thật (SQLi, XSS...) để xác nhận lỗ hổng; chỉ được làm trên hệ thống có quyền
False positive	Cảnh báo của scanner nhưng không phải lỗ hổng thật trong bối cảnh đang xét
Ground truth	Nhãn đúng do con người kiểm chứng thủ công, dùng làm chuẩn để chấm AI
Fingerprint	Mã định danh ngắn của một loại phát hiện, dùng để gom trùng, so sánh và làm khoá cache
Security header	Header HTTP yêu cầu trình duyệt bật cơ chế bảo vệ (CSP, HSTS, X-Frame-Options...)
Reverse proxy	Máy chủ đứng trước ứng dụng, nhận request thay ứng dụng (ví dụ nginx)
Ngăn xếp công nghệ (Stack)	Tổ hợp phần mềm máy chủ và framework mà target chạy trên đó
Structured output	Cơ chế buộc đầu ra của mô hình tuân theo một JSON schema định sẵn
Prompt	Chỉ dẫn và dữ liệu gửi cho mô hình ngôn ngữ
Token	Đơn vị văn bản mà mô hình xử lý; chi phí API tính theo số token vào và ra
Spider / AJAX Spider	Bộ dò đường dẫn của ZAP; AJAX Spider điều khiển trình duyệt thật để dò ứng dụng SPA
Bản vá (Patch)	Thay đổi cấu hình hoặc mã nguồn để loại bỏ lỗ hổng
Precision / Recall	Độ chính xác / độ bao phủ của một bộ phân loại (định nghĩa ở mục 2.7)

DANH MỤC TỪ VIẾT TẮT

Bảng 0.3 Bảng danh mục từ viết tắt

Từ viết tắt	Viết đầy đủ	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
CLI	Command-Line Interface	Giao diện dòng lệnh
CORS	Cross-Origin Resource Sharing	Cơ chế chia sẻ tài nguyên khác nguồn gốc
CSP	Content Security Policy	Chính sách giới hạn nguồn nội dung trang được tải
CWE	Common Weakness Enumeration	Danh mục điểm yếu phần mềm của MITRE
DAST	Dynamic Application Security Testing	Kiểm thử bảo mật ứng dụng khi đang chạy
DOM	Document Object Model	Mô hình đối tượng của trang web trong trình duyệt
FP	False Positive	Cảnh báo sai
HSTS	HTTP Strict Transport Security	Buộc trình duyệt chỉ dùng HTTPS
HTML / HTTP	HyperText Markup Language / Transfer Protocol	Ngôn ngữ trang web / giao thức web
JSON	JavaScript Object Notation	Định dạng dữ liệu
LLM	Large Language Model	Mô hình ngôn ngữ lớn
OWASP	Open Worldwide Application Security Project	Tổ chức mở về bảo mật ứng dụng
RFI	Remote File Inclusion	Chèn tệp từ xa
SAST	Static Application Security Testing	Kiểm thử bảo mật mã nguồn tĩnh
SPA	Single-Page Application	Ứng dụng web một trang
SQL / SQLi	Structured Query Language / SQL Injection	Ngôn ngữ truy vấn / lỗi chèn SQL
XSS	Cross-Site Scripting	Lỗi chèn mã script vào trang
ZAP	Zed Attack Proxy	Công cụ quét bảo mật web của OWASP
COOP / COEP / CORP	Cross-Origin Opener / Embedder / Resource Policy	Nhóm header cô lập nguồn gốc

DANH MỤC CÁC BẢNG, HÌNH ẢNH

Danh mục các bảng:

Bảng 0.1 Bảng phân công công việc và đánh giá thành viên.....	3
Bảng 0.2 Bảng danh mục thuật ngữ	5
Bảng 0.3 Bảng danh mục từ viết tắt	6
Bảng 1.1 Công nghệ sử dụng	14
Bảng 2.1 Mười nhóm rủi ro OWASP Top 10 bản 2021	17
Bảng 2.2 Các security header thường gặp	18
Bảng 2.3 Ba chế độ chạy ZAP trong đồ án	19
Bảng 2.4 Ma trận nhầm lẫn	20
Bảng 3.1 Các thư mục mã nguồn và vai trò	23
Bảng 3.2 Các trường của mô hình Finding	24
Bảng 3.3 Các bảng trong cơ sở dữ liệu	24
Bảng 3.4 Các trường của đối tượng Analysis.....	27
Bảng 3.5 Các endpoint của dashboard	28
Bảng 4.1 Các target trong lab	33
Bảng 4.2 Các lần quét được dùng trong báo cáo.....	33
Bảng 4.3 Nguồn của 115 cặp trong mẫu	34
Bảng 5.1 Số lần xuất hiện thô và số phát hiện sau gom trùng.....	38
Bảng 5.2 Lần quét #9 (trước vá) và #17 (sau vá) trên nginx.....	38
Bảng 5.3 Kiểm chứng bằng tay trên vulnapp trước khi quét lại	41
Bảng 5.4 Ma trận nhầm lẫn trên 115 cặp.....	43
Bảng 5.5 Các chỉ số chính, đợt 1 và gộp ba target	43
Bảng 5.6 Chỉ số theo từng target	44
Bảng 5.7 Khả năng false positive AI đưa ra, theo loại nhãn (gộp)	44
Bảng 5.8 Mức độ AI đánh giá, theo loại nhãn (gộp).....	44

Bảng 5.9 Tỷ lệ bản vá dùng được theo cách kiểm chứng	45
Bảng 5.10 Tỷ lệ bản vá dùng được theo target.....	45
Bảng 5.11 Chi phí API đo được	50

Danh mục hình ảnh:

Hình 3.1 Sơ đồ kiến trúc tổng quan của hệ thống	23
Hình 3.2 Trang chủ dashboard: form quét, biểu đồ diễn biến số lỗ hổng và lịch sử quét	29
Hình 3.3 Trang chi tiết lần quét #9: tóm tắt theo mức độ và thứ tự nên xử lý	30
Hình 3.4 Một phát hiện trên trang chi tiết: bằng chứng thô của scanner (trái) và phân tích, bản vá của AI (phải)	31
Hình 4.1 Hai quy trình đánh giá: (a) vòng lặp vá → quét lại, (b) đo độ chính xác bằng nhãn thủ công.....	34
Hình 5.1 Trang so sánh lần quét #9 và #17 trên nginx: 9 đã vá, 3 mới xuất hiện, 13 còn tồn tại	39
Hình 5.2 Trang so sánh lần quét #20 và #21 trên vulnapp: 17 đã vá, 2 mới xuất hiện, 2 còn tồn tại	42

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	2
BẢNG PHÂN CÔNG, ĐÁNH GIÁ THÀNH VIÊN	3
LỜI MỞ ĐẦU	4
DANH MỤC THUẬT NGỮ	5
DANH MỤC TỪ VIẾT TẮT	6
DANH MỤC CÁC BẢNG, HÌNH ẢNH	7
Chương 1: TỔNG QUAN ĐỀ TÀI	13
1.1 Đặt vấn đề và lý do chọn đề tài	13
1.2 Mục tiêu của đề tài	13
1.3 Phạm vi nghiên cứu	14
1.3.1 Phạm vi về đối tượng quét.....	14
1.3.2 Phạm vi về công nghệ.....	14
1.3.3 Giới hạn của đề tài.....	15
1.4 Đóng góp chính của đồ án	15
Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN	16
2.1 Kiểm thử bảo mật ứng dụng web	16
2.1.1 Quét thụ động và quét chủ động.....	16
2.1.2 Hạn chế của công cụ quét tự động.....	16
2.2 OWASP Top 10 và CWE	16
2.3 Các lỗ hổng và cấu hình được xét trong đồ án	17
2.3.1 SQL Injection	17
2.3.2 Cross-Site Scripting.....	17
2.3.3 Security header	17
2.3.4 CORS.....	18
2.3.5 Lộ thông tin	18

2.4 Công cụ Nikto.....	18
2.5 OWASP ZAP.....	19
2.6 Mô hình ngôn ngữ lớn trong phân tích bảo mật.....	19
2.6.1 Khả năng và rủi ro	19
2.6.2 Structured output	19
2.6.3 Token và chi phí	20
2.7 Chỉ số đánh giá bộ phân loại	20
2.8 Các nghiên cứu liên quan	20
Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	22
3.1 Yêu cầu hệ thống.....	22
3.1.1 Yêu cầu chức năng	22
3.1.2 Yêu cầu phi chức năng	22
3.2 Kiến trúc tổng quan	23
3.3 Hai hợp đồng dữ liệu.....	24
3.3.1 Mô hình Findings.....	24
3.3.2 Cơ sở dữ liệu SQLite.....	24
3.3.3 Fingerprint: một trường làm bốn việc	25
3.4 Tầng Scanner	25
3.4.1 Giới hạn thời gian cho full scan	26
3.4.2 Quét có đăng nhập	26
3.5 Tầng Analyzer	27
3.5.1 Schema đầu ra.....	27
3.5.2 Thiết kế prompt	27
3.5.3 Nhận diện ngăn xếp công nghệ	27
3.5.4 Chia lô, streaming và cache.....	28
3.5.5 Tóm tắt, thứ tự ưu tiên và trường hợp bị từ chối.....	28
3.6 Backend và giao diện.....	28

3.7 Các biện pháp an toàn của chính hệ thống	31
3.8 Kiểm thử phần mềm	32
Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ	33
4.1 Môi trường thực nghiệm.....	33
4.2 Vòng lặp vá → quét lại.....	34
4.3 Xây dựng ground truth	34
4.3.1 Mẫu	34
4.3.2 Quy tắc gán nhãn	35
4.3.3 Kiểm soát thiên lệch	35
4.3.4 Nhật ký chỉnh sửa phiếu	36
4.4 Chỉ số đánh giá	37
Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	38
5.1 Giảm khối lượng đọc thủ công	38
5.2 Vòng lặp vá trên nginx (tầng cấu hình)	38
5.2.1 Bản vá của AI sai ở đâu.....	40
5.3 Vòng lặp vá trên vulnapp (tầng mã nguồn)	40
5.3.1 Bản vá của AI sai ở đâu.....	43
5.4 Độ chính xác khi nhận diện false positive.....	43
5.4.1 Recall thấp đến từ đâu	44
5.4.2 Độ nhạy theo ngưỡng	45
5.5 Chất lượng bản vá.....	45
5.6 AI đánh giá lại mức độ nghiêm trọng.....	46
5.7 Các ca đáng thảo luận.....	46
5.8 Quét có đăng nhập trên Juice Shop	48
5.9 Lỗi của chính công cụ do vòng lặp và target thứ ba phát hiện.....	48
5.10 Target bị chính active scan làm hỏng.....	49
5.11 Chi phí vận hành.....	50

Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	51
6.1 Kết quả đạt được.....	51
6.2 Bài học rút ra	51
6.3 Hạn chế	52
6.4 Hướng phát triển.....	52
TÀI LIỆU THAM KHẢO	54
PHỤ LỤC: HƯỚNG DẪN CHẠY LẠI.....	56

Chương 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề và lý do chọn đề tài

Theo OWASP Top 10 bản 2021 [1], các nhóm rủi ro phổ biến nhất của ứng dụng web gồm kiểm soát truy cập hỏng, lỗi mã hoá, injection và cấu hình bảo mật sai. Nhiều lỗi trong số đó, đặc biệt là lỗi cấu hình, có thể phát hiện tự động bằng các công cụ kiểm thử động (DAST) như Nikto [10] và OWASP ZAP [9]. Đây là những công cụ mã nguồn mở, miễn phí và được dùng rộng rãi trong giảng dạy lẫn thực tế.

Tuy nhiên, khi chạy thử trên chính lab của nhóm, đầu ra của hai công cụ này bộc lộ ba vấn đề đo được cụ thể:

- **Nhiều và trùng lặp.** Một lần quét nginx đặt trước OWASP Juice Shop sinh ra 200 lần xuất hiện cảnh báo. Riêng một test của Nikto về file sao lưu đã khớp 140 URL khác nhau, trong khi thực chất chỉ là một vấn đề với một cách vá.
- **Lấn cảnh báo không đáng lo.** Trong 115 phát hiện nhóm kiểm chứng thủ công trên ba target, 51 không phải lỗ hổng thật trong bối cảnh lab: 11 là scanner khẳng định sai, 40 là quan sát đúng nhưng không gây rủi ro.
- **Không nói phải sửa gì, ở tầng nào.** Gợi ý gốc của ZAP cho lỗi thiếu CSP chỉ là “Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header”. Câu này đúng với mọi hệ thống nên không giúp được hệ thống cụ thể nào.

Mô hình ngôn ngữ lớn có khả năng đọc hiểu văn bản kỹ thuật, giải thích bằng ngôn ngữ tự nhiên và sinh mã. Về lý thuyết, LLM có thể lấp đúng ba khoảng trống trên. Nhưng LLM cũng có thể đưa ra câu trả lời sai với giọng rất tự tin, và các nghiên cứu về dùng LLM để vá lỗ hổng [6], [8] đều cho thấy chất lượng không ổn định. Vì vậy, một hệ thống “AI hỗ trợ quét lỗ hổng” chỉ có giá trị khi đi kèm cách đo xem AI nói đúng đến đâu. Đó là lý do nhóm chọn đề tài này và đặt phân đánh giá làm trọng tâm.

1.2 Mục tiêu của đề tài

Hệ thống phải trả lời được ba câu hỏi mà scanner thuần không trả lời được:

1. **Cái nào đáng lo thật?** AI đánh giá lại mức độ nghiêm trọng và ước lượng khả năng một cảnh báo là false positive.

2. **Sửa thế nào?** AI sinh bản vá cụ thể (đoạn cấu hình hoặc mã) đúng tăng công nghệ của mục tiêu.

3. **AI nói có đúng không?** Nhóm đo bằng ground truth gán nhãn thủ công, và chứng minh bằng vòng lặp áp bản vá → quét lại → lỗ hổng biến mất.

Cụ thể, đề án đặt ra các mục tiêu sau:

- Xây dựng hệ thống chạy Nikto và OWASP ZAP qua Docker, chuẩn hoá và gom trùng kết quả, rồi gửi cho mô hình Claude phân tích theo một schema cố định.
- Cung cấp cả giao diện dòng lệnh lẫn dashboard web: chạy quét, theo dõi tiến trình, xem lịch sử, so sánh hai lần quét và xuất báo cáo HTML.
- Dựng môi trường lab gồm một target vá được ở tầng cấu hình (nginx) và một target vá được ở tầng mã nguồn (ứng dụng Flask do nhóm viết), rồi chạy vòng lặp vá → quét lại trên cả hai; thêm OWASP Mutillidae II làm target thứ ba để mở rộng mẫu đánh giá AI.
- Đo định lượng: precision, recall của AI khi nhận diện false positive; tỉ lệ bản vá dùng được; mức giảm khối lượng đọc thủ công; chi phí API.

1.3 Phạm vi nghiên cứu

1.3.1 Phạm vi về đối tượng quét

Đề án chỉ quét các hệ thống nhóm tự dựng trên máy cá nhân: OWASP Juice Shop [11], một reverse proxy nginx đặt trước Juice Shop, ứng dụng Flask “vulnapp” do nhóm tự viết, và OWASP Mutillidae II [16] làm target thứ ba cho phần đánh giá AI. Quét hệ thống của người khác khi chưa được phép là vi phạm pháp luật. Luật An ninh mạng 2018 [17] và Bộ luật Hình sự (Điều 289, tội xâm nhập trái phép vào mạng máy tính, mạng viễn thông hoặc phương tiện điện tử của người khác) [18] đều có quy định liên quan. Vì vậy hệ thống mặc định chỉ chấp nhận mục tiêu là localhost hoặc địa chỉ mạng nội bộ (RFC 1918), chặn ở cả giao diện dòng lệnh lẫn web (mục 3.7).

1.3.2 Phạm vi về công nghệ

Bảng 1.1 Công nghệ sử dụng

Thành phần	Công nghệ
Scanner	Nikto (image ghcr.io/sullo/nikto), OWASP ZAP (ghcr.io/zaproxy/zaproxy:stable), chạy qua Docker Desktop
Mô hình AI	Claude (claude-opus-5) qua Anthropic API, dùng structured output và streaming

Thành phần	Công nghệ
Ngôn ngữ	Python 3.14
Backend / giao diện	FastAPI, Jinja2 (bật autoescape), HTMX (đóng gói sẵn trong dự án)
Lưu trữ	SQLite qua thư viện chuẩn <code>sqlite3</code> , không dùng ORM
Lab	OWASP Juice Shop, nginx:alpine, Flask 3.1.3, OWASP Mutillidae II
Kiểm thử	pytest, 59 test chạy hoàn toàn offline

1.3.3 Giới hạn của đề tài

- Chỉ kiểm thử hộp đen: AI chỉ đọc kết quả của scanner, không đọc mã nguồn của target.
- Không sinh mã khai thác. Active scan chỉ chạy trong phạm vi ZAP full scan, trên target của nhóm.
- Hệ thống chạy cục bộ trên một máy, không có đăng nhập, không phục vụ nhiều người dùng.
- Ground truth do hai người gán chia nhau các dòng (không gán trùng), cỡ mẫu 115 cặp (phát hiện, target), nên các chỉ số mang tính chỉ dẫn, chưa đủ để khái quát.

1.4 Đóng góp chính của đề án

- Một pipeline chạy thông từ đầu đến cuối: quét bằng hai scanner → gom trùng → AI phân tích và sinh bản vá → lưu lịch sử → so sánh trước/sau khi vá, dùng được qua CLI lẫn dashboard.
- Thiết kế **fingerprint** một trường làm bốn việc: gom trùng, so sánh hai lần quét, khoá cache kết quả AI và khoá nối với ground truth.
- Bộ đánh giá có kiểm soát thiên lệch: phiếu gán nhãn không chứa kết luận của AI, nhãn được chốt trước khi xem số liệu, mọi chỉnh sửa sau đó đều ghi nhận ký.
- Kết quả đo được trên 115 cặp của ba target: precision 86,7% (13/15), recall 25,5% (13/51); 87% bản vá dùng được (13/15); vòng lặp vá xoá 9/22 phát hiện trên nginx và 17/19 phát hiện trên vulnapp, gồm cả ba lỗ hổng High.
- Vòng lặp vá target thứ ba làm lộ ra bốn lỗi của chính công cụ mà test đơn vị không bắt được. Ba lỗi đã được sửa kèm test hồi quy; một lỗi được ghi thành hạn chế (mục 5.9).
- Một bài học an toàn đo được trên thực tế: active scan đi qua phpMyAdmin không mật khẩu đã chạy SQL thật và làm hỏng chính target Mutillidae (mục 5.10).

Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 Kiểm thử bảo mật ứng dụng web

Hướng dẫn NIST SP 800-115 [3] chia kỹ thuật đánh giá an ninh thành ba nhóm: rà soát tài liệu và cấu hình, xác định và phân tích mục tiêu (trong đó có quét lỗ hổng), và xác thực lỗ hổng (kiểm thử xâm nhập). Với ứng dụng web, có hai cách tiếp cận tự động phổ biến. Kiểm thử tĩnh (SAST) đọc mã nguồn. Kiểm thử động (DAST) coi ứng dụng như hộp đen: gửi request HTTP và suy luận từ phản hồi. Nikto và ZAP đều thuộc nhóm DAST, nên dùng được cả khi không có mã nguồn, như trường hợp Juice Shop trong đồ án.

2.1.1 Quét thụ động và quét chủ động

Quét thụ động chỉ quan sát các phản hồi bình thường của ứng dụng: header thiếu hoặc sai, cookie thiếu cờ bảo vệ, lộ phiên bản phần mềm. Cách này an toàn và nhanh. Quét chủ động gửi payload tấn công thật (chuỗi SQL injection, đoạn script XSS...) để xác nhận lỗ hổng. Cách này mạnh hơn nhưng có thể làm thay đổi dữ liệu của hệ thống, nên chỉ được chạy trên hệ thống có quyền. Trong đồ án, SQL injection của vulnapp chỉ được ZAP phát hiện khi bật active scan (profile `full`).

2.1.2 Hạn chế của công cụ quét tự động

Hai hạn chế đã được nghiên cứu từ lâu là độ bao phủ khi dò đường dẫn (crawling) và tỉ lệ cảnh báo sai [4], [5]. Với ứng dụng một trang (SPA) như Juice Shop, spider truyền thống chỉ thấy được rất ít đường dẫn, vì nội dung được JavaScript sinh ra sau khi tải trang. Ngoài ra scanner không hiểu bối cảnh: một header CORS mở trên API công khai và trên API trả dữ liệu cá nhân được báo giống hệt nhau. Đây chính là khoảng trống mà đồ án thử lấp bằng AI.

2.2 OWASP Top 10 và CWE

OWASP Top 10 [1] là tài liệu nhận thức về mười nhóm rủi ro hàng đầu của ứng dụng web, được dùng làm ngôn ngữ chung giữa người phát triển và người kiểm thử. CWE [2] là danh mục điểm yếu phần mềm do MITRE duy trì, mỗi điểm yếu có một mã số (ví dụ CWE-89 là SQL injection, CWE-79 là XSS). ZAP gắn sẵn mã CWE cho

mỗi cảnh báo. AI trong đồ án được yêu cầu ánh xạ mỗi phát hiện sang cả OWASP Top 10 lẫn CWE.

Bảng 2.1 Mười nhóm rủi ro OWASP Top 10 bản 2021

Mã	Tên nhóm rủi ro
A01:2021	Broken Access Control – Kiểm soát truy cập hỏng
A02:2021	Cryptographic Failures – Lỗi mã hoá
A03:2021	Injection – Chèn mã (SQL injection, XSS...)
A04:2021	Insecure Design – Thiết kế không an toàn
A05:2021	Security Misconfiguration – Cấu hình bảo mật sai
A06:2021	Vulnerable and Outdated Components – Thành phần lỗi thời, có lỗ hổng
A07:2021	Identification and Authentication Failures – Lỗi định danh và xác thực
A08:2021	Software and Data Integrity Failures – Lỗi toàn vẹn phần mềm và dữ liệu
A09:2021	Security Logging and Monitoring Failures – Thiếu ghi log và giám sát
A10:2021	Server-Side Request Forgery (SSRF)

2.3 Các lỗ hổng và cấu hình được xét trong đồ án

2.3.1 SQL Injection

SQL injection (CWE-89, thuộc A03) xảy ra khi dữ liệu người dùng được ghép thẳng vào câu truy vấn. Kẻ tấn công chèn cú pháp SQL để đổi ý nghĩa câu lệnh, ví dụ đọc toàn bộ bảng người dùng. Cách phòng chống chuẩn là truy vấn tham số hoá (prepared statement), kết hợp không trả lỗi cơ sở dữ liệu chi tiết về cho người dùng (CWE-209) [21].

2.3.2 Cross-Site Scripting

XSS (CWE-79) xảy ra khi dữ liệu người dùng được nhúng vào trang mà không được mã hoá, khiến trình duyệt của nạn nhân chạy script của kẻ tấn công. Có ba dạng. *Reflected*: payload nằm trong request và được phản chiếu ngay. *Stored*: payload được lưu lại rồi hiển thị cho người khác. *DOM-based*: mã JavaScript phía trình duyệt tự đưa dữ liệu không tin cậy vào DOM. Cách phòng chống chính là mã hoá đầu ra theo ngữ cảnh (các template engine như Jinja2 tự làm khi bật autoescape), bổ sung thêm lớp CSP [13], [21].

2.3.3 Security header

Phần lớn cảnh báo của cả hai scanner trên lab là về header HTTP. Đây cũng là nhóm vá được ở tầng cấu hình mà không cần sửa mã ứng dụng [12].

Bảng 2.2 Các security header thường gặp

Header	Tác dụng
Content-Security-Policy	Giới hạn nguồn script, style, ảnh, iframe... mà trang được tải; lớp phòng thủ thứ hai trước XSS
X-Frame-Options / frame-ancestors	Chặn trang bị nhúng vào iframe của site khác (chống clickjacking)
X-Content-Type-Options: nosniff	Cấm trình duyệt tự đoán kiểu nội dung
Strict-Transport-Security	Buộc trình duyệt chỉ dùng HTTPS; vô nghĩa trên HTTP thuần
Referrer-Policy	Giới hạn thông tin URL gửi kèm khi chuyển trang
Permissions-Policy	Tắt các tính năng trình duyệt không dùng (camera, micro, định vị...)
Cross-Origin-Opener / Embedder / Resource-Policy	Cô lập trang khỏi cửa sổ và tài nguyên khác nguồn gốc

2.3.4 CORS

CORS [14] cho phép máy chủ nói lòng chính sách cùng nguồn gốc. Header `Access-Control-Allow-Origin: *` cho phép mọi trang web đọc phản hồi. Tuy nhiên, với giá trị `*`, trình duyệt không cho trang khác đọc phản hồi của request có kèm thông tin xác thực. Vì vậy mức rủi ro phụ thuộc vào việc endpoint trả dữ liệu công khai hay dữ liệu riêng. Ca này được thảo luận kỹ ở mục 5.7.

2.3.5 Lộ thông tin

Nhóm này gồm banner `Server` lộ tên và phiên bản phần mềm, thư mục liệt kê được nội dung (directory listing), file sao lưu còn sót lại, và mục trong `robots.txt` trở tới vùng nhạy cảm. Bản thân từng mục thường ở mức thấp, nhưng chúng giúp kẻ tấn công thu hẹp phạm vi dò tìm.

2.4 Công cụ Nikto

Nikto [10] là công cụ quét máy chủ web mã nguồn mở. Nikto kiểm tra file và chương trình nguy hiểm, phiên bản phần mềm lỗi thời và các vấn đề cấu hình, dựa trên một cơ sở dữ liệu test. Mỗi test có một mã định danh. Hai đặc điểm của Nikto ảnh hưởng trực tiếp tới thiết kế của đồ án. Thứ nhất, Nikto **không xếp hạng mức độ nghiêm trọng**, nên mọi phát hiện đều ở mức Info. Thứ hai, Nikto **dùng chung một mã test cho cả họ phát hiện**: mọi header lạ, hay mọi header bảo mật bị thiếu, đều mang cùng một mã. Đồ án dùng image Docker chính thức `ghcr.io/sullo/nikto` với đầu ra JSON.

2.5 OWASP ZAP

OWASP ZAP [9] là công cụ kiểm thử bảo mật web mã nguồn mở hoạt động như một proxy. ZAP có spider truyền thống, AJAX Spider (điều khiển trình duyệt thật để dò ứng dụng SPA), bộ luật quét thụ động và bộ luật quét chủ động. Mỗi cảnh báo có `pluginId` (luật sinh ra nó), `alertRef` (biên thể cụ thể của luật), mức rủi ro, độ tin cậy và mã CWE. Đồ án dùng ba cách chạy:

Bảng 2.3 Ba chế độ chạy ZAP trong đồ án

Profile	Cách chạy	Đặc điểm
baseline	<code>zap-baseline.py</code>	Spider + quét thụ động; nhanh, an toàn; không phát hiện SQLi
full	<code>zap-full-scan.py</code>	Thêm quét chủ động; lâu hơn nhiều; được giới hạn thời gian (mục 3.4)
auth	<code>zap.sh -autorun</code> với kế hoạch Automation Framework	Đăng nhập trước rồi mới quét, dùng AJAX Spider; chạm được phần sau màn đăng nhập

Một lưu ý kỹ thuật: các script đóng gói của ZAP trả exit code khác 0 khi *tìm thấy* cảnh báo. Vì vậy exit code khác 0 không có nghĩa là lỗi. Hệ thống coi lần chạy thành công khi file JSON kết quả tồn tại và đọc được.

2.6 Mô hình ngôn ngữ lớn trong phân tích bảo mật

2.6.1 Khả năng và rủi ro

LLM là mô hình học sâu được huấn luyện trên lượng văn bản rất lớn, có khả năng đọc hiểu, tóm tắt, giải thích và sinh mã nguồn. Trong bảo mật, LLM được thử nghiệm để giải thích cảnh báo, hỗ trợ kiểm thử xâm nhập [7] và sửa lỗ hổng [6], [8]. Rủi ro chính là mô hình có thể sinh nội dung sai nhưng trình bày rất thuyết phục (hallucination), ví dụ bịa ra tên file hay tên bảng không tồn tại. Rủi ro này được đo trực tiếp trong chương 5.

2.6.2 Structured output

Đề dùng đầu ra của mô hình như dữ liệu, đồ án khai báo một schema (bằng Pydantic) và yêu cầu API trả về đúng schema đó [15]. Khi API bảo đảm đầu ra khớp schema, phía ứng dụng không cần tự bóc tách JSON từ văn bản, cũng không cần thử lại khi JSON hỏng. Điều này quan trọng với phân đánh giá. Nếu một số phân tích rơi mất vì JSON hỏng, các phân tích rơi thường rơi vào những phát hiện có bằng chứng

dài, ký tự lạ, tức đúng những ca khó. Khi đó số liệu precision/recall bị lệch một cách có hệ thống.

2.6.3 Token và chi phí

API tính phí theo số token đầu vào và đầu ra. Trong đồ án, mỗi phát hiện được trả về kèm giải thích, các bước vá và đoạn mã vá. Vì vậy phần lớn chi phí nằm ở token *đầu ra*: đo được khoảng 2.277 token ra cho mỗi phát hiện. Kiến trúc cache (mục 3.5) được thiết kế để mỗi phát hiện chỉ phải trả tiền phân tích một lần cho mỗi target.

2.7 Chỉ số đánh giá bộ phân loại

Việc AI quyết định một cảnh báo có phải false positive hay không được xem là một bài toán phân loại nhị phân. Để tránh nhầm với chữ “false positive” của scanner, báo cáo gọi lớp cần phát hiện là **lớp dương** = “cảnh báo này không phải lỗ hổng thật”. Từ ma trận nhầm lẫn:

Bảng 2.4 Ma trận nhầm lẫn

	AI nói: không phải lỗ hổng	AI nói: lỗ hổng thật
Thật sự không phải lỗ hổng	Dương đúng (TP)	Âm sai (FN)
Thật sự là lỗ hổng	Dương sai (FP)	Âm đúng (TN)

- **Precision** = $TP / (TP + FP)$: trong những cảnh báo AI bảo bỏ qua, bao nhiêu phần đúng là bỏ qua được. Precision thấp nghĩa là AI bác nhầm lỗ hổng thật, loại lỗi nguy hiểm nhất với công cụ bảo mật.
- **Recall** = $TP / (TP + FN)$: trong những cảnh báo thật sự bỏ qua được, AI nhận ra bao nhiêu. Recall thấp chỉ làm người dùng tốn thời gian đọc.
- **Accuracy** = $(TP + TN) / \text{tổng số}$: tỉ lệ quyết định đúng chung.

2.8 Các nghiên cứu liên quan

Đánh giá scanner hộp đen. Doupé và cộng sự [4] thử 11 scanner web hộp đen trên một ứng dụng có ý chứa lỗ hổng. Kết quả cho thấy scanner bỏ sót nhiều lớp lỗ hổng, và khả năng dò đường dẫn là điểm nghẽn lớn. Bau và cộng sự [5] đánh giá tám scanner thương mại, kết luận các công cụ phát hiện khá tốt những lỗi quen thuộc như XSS phản chiếu nhưng yếu với các lỗi cần hiểu trạng thái ứng dụng.

LLM sửa lỗ hổng. Pearce và cộng sự [6] thử dùng LLM sửa lỗ hổng ở chế độ zero-shot. Mô hình sửa được nhiều kịch bản tổng hợp, nhưng kết quả trên lỗi thật hạn

chế hơn và nhạy với cách viết prompt. Fu và cộng sự [8] so ChatGPT với các mô hình chuyên biệt trong các tác vụ phát hiện, phân loại và sửa lỗi hổng, và thấy ChatGPT còn thua các mô hình chuyên biệt.

LLM hỗ trợ kiểm thử xâm nhập. PentestGPT [7] cho thấy LLM làm tốt các bước con (đọc output công cụ, đề xuất bước tiếp theo) nhưng khó giữ được bối cảnh tổng thể của cả phiên kiểm thử.

Điểm khác của đồ án. Đồ án không huấn luyện mô hình, cũng không để AI tự tấn công. AI chỉ đứng sau hai scanner có sẵn để hậu xử lý kết quả. Phần đóng góp nằm ở cách đo: nhân thủ công được kiểm soát thiên lệch, và bản vá được kiểm chứng bằng cách quét lại chứ không chỉ đọc bằng mắt.

Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 Yêu cầu hệ thống

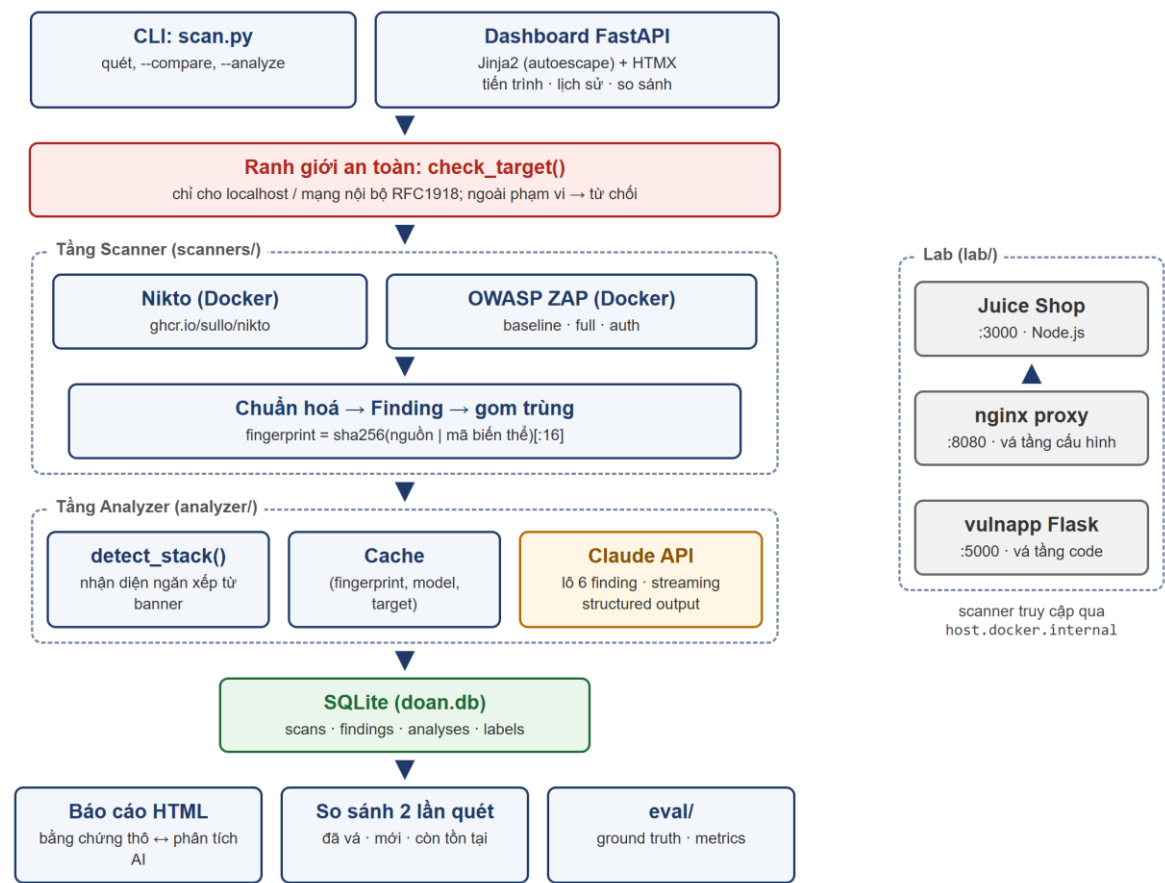
3.1.1 Yêu cầu chức năng

- Nhận URL mục tiêu, kiểm tra phạm vi được phép, chạy Nikto và ZAP song song.
- Chuẩn hoá kết quả hai scanner về một mô hình chung, gom trùng.
- Gửi các phát hiện cho AI phân tích: đánh giá lại mức độ, khả năng false positive, giải thích tiếng Việt, ánh xạ OWASP/CWE, các bước vá, đoạn mã vá, cách kiểm chứng.
- Lưu lịch sử quét, so sánh hai lần quét thành ba nhóm: đã vá, mới xuất hiện, còn tồn tại.
- Xuất báo cáo HTML tự chứa, đặt bằng chứng thô của scanner cạnh phân tích của AI.
- Xuất phiếu gán nhãn, thu bằng chứng, tính chỉ số đánh giá.

3.1.2 Yêu cầu phi chức năng

- **An toàn:** không quét ngoài phạm vi; không lưu khoá API hay mật khẩu trong mã nguồn; báo cáo không được tự tạo lỗ hổng XSS.
- **Chi phí:** không trả tiền hai lần cho cùng một phân tích; một lỗi không làm mất các lỗi đã trả.
- **Tái lập được:** mọi con số trong báo cáo đều chạy lại được bằng lệnh có ghi trong tài liệu.
- **Chạy được trên Windows:** xử lý các bẫy về đường dẫn có dấu cách và mạng Docker.
- **Kiểm thử offline:** test không cần Docker, mạng hay API.

3.2 Kiến trúc tổng quan



Hình 3.1 Sơ đồ kiến trúc tổng quan của hệ thống

Hệ thống có hai cửa vào là CLI `scan.py` và dashboard FastAPI. Cả hai gọi chung một bộ hàm và dùng chung một cơ sở dữ liệu, nên một lần quét chạy bằng CLI vẫn xem được trên dashboard và ngược lại. Mọi URL đều phải qua `check_target()` trước khi tới tầng scanner. Tầng scanner chạy hai container song song, chuẩn hoá kết quả về mô hình `Finding` và gom trùng theo fingerprint. Tầng analyzer nhận diện ngăn xếp công nghệ, lấy phân tích có sẵn từ cache, và chỉ gửi phần còn thiếu lên API theo từng lô. Kết quả được ghi vào SQLite, từ đó sinh báo cáo, trang so sánh và dữ liệu cho phân đánh giá.

Bảng 3.1 Các thư mục mã nguồn và vai trò

Thư mục / file	Vai trò
core/	Hai “hợp đồng” dùng chung: mô hình <code>Finding</code> (<code>models.py</code>) và schema SQLite (<code>db.py</code>); cấu hình và ranh giới an toàn (<code>config.py</code>)
scanners/	Chạy Nikto, ZAP qua Docker; đọc kết quả; gom trùng; chạy song song (<code>runner.py</code>)
analyzer/	Prompt, schema đầu ra của AI, gọi API theo lô, cache, nhận diện ngăn xếp
api/, web/	Dashboard FastAPI, template Jinja2, HTMX

Thư mục / file	Vai trò
report.py	Dựng báo cáo HTML tự chứa
lab/	Target lab: cấu hình nginx, ứng dụng vulnapp, bản chứa vá để làm lại từ đầu
eval/	Xuất phiếu gán nhãn, thu bằng chứng, tính chỉ số, ghi kết quả
tests/	59 test offline với dữ liệu mẫu JSON của Nikto và ZAP

3.3 Hai hợp đồng dữ liệu

Hệ thống được chia module để nhiều người làm song song. Điều đó chỉ khả thi nếu hai cấu trúc dữ liệu dùng chung được chốt từ đầu và ổn định. Mọi module khác chỉ trao đổi với nhau qua hai cấu trúc này.

3.3.1 Mô hình Finding

Bảng 3.2 Các trường của mô hình Finding

Trường	Ý nghĩa
fingerprint	Mã định danh loại phát hiện (mục 3.3.3)
source	nikto hoặc zap
name	Tên cảnh báo của scanner
severity	Mức của scanner: High, Medium, Low, Info
urls	Tối đa 5 URL mẫu
count	Số lần xuất hiện thật trước khi gom
evidence, description	Bằng chứng và mô tả gốc của scanner
solution_raw	Gợi ý sửa gốc của scanner, giữ lại để đối chiếu với AI
cwe	Mã CWE (ZAP có sẵn)

3.3.2 Cơ sở dữ liệu SQLite

Bảng 3.3 Các bảng trong cơ sở dữ liệu

Bảng	Nội dung chính	Khoá
scans	Mục tiêu, profile, thời gian, trạng thái, lỗi, tóm tắt, thứ tự ưu tiên	id
findings	Các phát hiện đã chuẩn hoá của từng lần quét	id, liên kết scan_id
analyses	Cache phân tích của AI (JSON)	(fingerprint, model, target)
labels	Dự phòng cho nhãn; ground truth dùng trong đánh giá nằm ở eval/ground_truth.csv, khoá (fingerprint, target)	fingerprint

Bảng analyses không khoá theo lần quét mà khoá theo (fingerprint, model, target). Nhờ vậy, khi quét lại cùng target, những phát hiện không đổi được lấy thẳng từ cache mà không gọi lại API. Có model trong khoá để khi đổi phiên bản

mô hình thì hệ thống phân tích lại, thay vì lặng lẽ dùng kết quả của mô hình cũ. Có `target` trong khoá vì cùng một lỗi nhưng bản vá phụ thuộc ngăn xếp. Lý do này được phát hiện qua một lỗi thật, trình bày ở mục 5.9.

3.3.3 Fingerprint: một trường làm bốn việc

`fingerprint = sha256(nguồn | mã biến thể)`, cắt còn 16 ký tự. Trường này được dùng để:

1. gom trùng các cảnh báo trong một lần quét;
2. so sánh hai lần quét: phép so sánh chỉ là phép toán trên tập hợp fingerprint;
3. làm khoá cache kết quả AI (ghép thêm model và target);
4. làm khoá nối với ground truth (ghép thêm target).

Ba quyết định thiết kế về “mã biến thể” đều xuất phát từ dữ liệu đo được:

- **URL cố ý không nằm trong khoá.** Test “backup/cert file found” của Nikto khớp 140 URL khác nhau trên Juice Shop. Nếu tính cả URL thì báo cáo có 140 “lỗi hỏng”, trong khi thực chất chỉ là một vấn đề với một cách vá. Báo cáo sẽ không đọc nổi, và lô gửi lên API vượt giới hạn token. Số URL bị ảnh hưởng vẫn được giữ trong `count` và `urls`.
- **ZAP dùng `alertRef` chứ không dùng `pluginId`.** Một luật của ZAP phát ra nhiều biến thể khác hẳn nhau. “CSP: Failure to Define Directive with No Fallback” và “CSP: script-src unsafe-inline” đều thuộc luật 10055. Nếu gom theo `pluginId`, trang so sánh sẽ báo “không đổi” trong khi vấn đề đã khác, và bằng chứng “đã vá” trở thành sai.
- **Với Nikto, khoá gồm mã test cộng phần nằm trong dấu nháy đơn và tên header sau chữ “missing:”.** Nikto dùng chung một mã test cho mọi header lạ và mọi header bảo mật bị thiếu. Nếu chỉ dùng mã test, năm header thiếu bị gộp làm một: vá bốn cái vẫn báo “còn tồn tại”. Mỗi ca đều có test hồi quy.

3.4 Tầng Scanner

Mỗi scanner chạy trong một container Docker tạm (`docker run --rm`), đầu ra JSON được ghi vào một thư mục tạm. Hai scanner chạy song song bằng `ThreadPoolExecutor`. Nếu một scanner lỗi thì lần quét vẫn tiếp tục với scanner còn lại, kèm cảnh báo. Các bất kỳ riêng của Windows được xử lý từ đầu:

- Đường dẫn dự án có dấu cách (IT Learning, Cyber security) làm bind mount của Docker dễ hỏng, nên đầu ra được ghi vào %TEMP%.
- Container không thấy localhost của máy host. Hệ thống đổi localhost thành host.docker.internal khi truyền vào container, nhưng vẫn giữ URL gốc để hiển thị.
- Scanner trả exit code khác 0 khi tìm thấy cảnh báo. Điều kiện thành công là file JSON tồn tại và đọc được.

3.4.1 Giới hạn thời gian cho full scan

Mặc định, active scan của zap-full-scan.py không giới hạn thời gian, còn tăng runner cắt tiến trình ở 30 phút. Bị runner cắt thì mất trắng kết quả. Nếu để ZAP tự dừng thì vẫn có báo cáo cho phần đã quét. Vì vậy profile full giới hạn spider tối đa 5 phút, active scan tối đa 15 phút và mỗi luật tối đa 3 phút. Phần thời gian còn lại dành cho khởi động và quét thụ động.

3.4.2 Quét có đăng nhập

Profile auth sinh một kế hoạch Automation Framework từ template. Tài khoản đọc từ biến môi trường, không lưu trong mã nguồn, kể cả tài khoản lab. Điểm thiết kế đáng chú ý là **đăng nhập được kiểm tra trước khi khởi động ZAP**. Nhóm đã thử để ZAP tự kiểm tra bằng job requestor với failOnError: true. Khi chạy thật với mật khẩu sai, ZAP chỉ coi việc lệch mã phản hồi là cảnh báo: nó vẫn quét hết và vẫn sinh báo cáo. Kết quả là một lần quét ả danh trông y hệt lần quét có đăng nhập. Với một đề án lấy bằng chứng làm trung tâm, đây là kiểu hỏng tệ nhất vì không có lỗi nào báo ra.

Vì vậy công cụ tự đăng nhập thử, lấy token, rồi gọi một endpoint cần quyền. Endpoint này phải trả 401 khi chưa đăng nhập: với Juice Shop là /api/Cards, còn /rest/user/whoami không dùng được vì trả 200 trong cả hai trường hợp. Nếu kiểm tra hỏng, công cụ dừng ngay kèm thông báo rõ ràng; ZAP bị bỏ qua còn Nikto vẫn chạy.

3.5 Tầng Analyzer

3.5.1 Schema đầu ra

Mỗi phát hiện nhận về một đối tượng `Analysis`, và mỗi lần gọi trả về một `ReportOut`:

Bảng 3.4 Các trường của đối tượng `Analysis`

Trường	Nội dung
<code>severity_ai</code>	Mức độ AI đánh giá lại: Critical, High, Medium, Low, Info
<code>false_positive_risk</code>	Khả năng là false positive: Cao, Trung bình, Thấp
<code>explain_vi</code>	Giải thích lỗ hổng bằng tiếng Việt cho người mới học
<code>impact_vi</code>	Kẻ tấn công làm được gì nếu khai thác thành công
<code>owasp_top10, cwe</code>	Ánh xạ sang OWASP Top 10 2021 và CWE
<code>fix_steps</code>	Các bước vá theo thứ tự
<code>fix_snippet</code>	Đoạn cấu hình/mã dán được ngay; rỗng nếu không có
<code>verify_vi</code>	Cách kiểm chứng đã vá xong

3.5.2 Thiết kế prompt

Prompt hệ thống nêu rõ bối cảnh: mục tiêu là lab do nhóm tự dựng, mục đích là phòng thủ và vá lỗi, không sinh mã khai thác. Các yêu cầu chính:

- Đánh giá lại mức độ theo rủi ro thực tế, vì Nikto không xếp hạng còn ZAP hay thói phòng các vấn đề về header.
- Trung thực về false positive: nếu bằng chứng yếu (suy đoán từ banner, từ danh sách file phổ biến) thì phải nói là yếu.
- **Không đoán tầng công nghệ**, mà suy ra từ bằng chứng (banner `Server`, header, đường dẫn). Nếu không đủ bằng chứng thì phải nêu rõ giả định ở dòng đầu của `fix_steps`.
- Bản vá phải đúng tầng mà người vận hành kiểm soát được: có reverse proxy thì vá ở proxy, lỗi logic (SQLi, XSS) thì vá trong mã bằng đúng framework.
- `fix_snippet` phải dán được ngay, không viết chung chung kiểu “hãy cấu hình header cho đúng”.

3.5.3 Nhận diện ngăn xếp công nghệ

Ở lần chạy đầu, mô hình vá sai tầng: nó trả về `helmet` của Express (Node.js) cho một mục tiêu chạy nginx. Nguyên nhân là trong năm phát hiện về header, chỉ đúng một phát hiện có chuỗi `nginx/1.31.5` trong bằng chứng. Bốn phát hiện còn lại không

có manh mối nào, nên mô hình đoán. Cách sửa là hàm `detect_stack()`: quét banner trên **toàn bộ** lần quét (nhận biết nginx, Apache, Werkzeug, Python, PHP, IIS, Express), rồi đưa kết quả vào đầu mỗi lô yêu cầu. Sau khi sửa, cả 4/4 phát hiện sinh lại đều trả về `add_header/proxy_hide_header` của nginx.

3.5.4 Chia lô, streaming và cache

Gộp cả 22 phát hiện vào một lần gọi thì chạm trần `max_tokens`, JSON bị cắt giữa chừng và mất toàn bộ. Đo được khoảng 2.277 token đầu ra cho mỗi phát hiện, nên hệ thống chia lô 6 phát hiện (khoảng 13.700 token, còn dư gần nửa so với trần 32.000). Mỗi lô được gọi bằng `messages.stream` với `output_format=ReportOut`. Phải dùng stream vì SDK từ chối lời gọi thường ở mức `max_tokens` này. Lô nào xong được ghi cache ngay. Nếu một lô lỗi, lô đó không kéo theo các lô khác, và lô đã trả tiền không bao giờ phải trả lại.

Trước khi gọi API, hệ thống tra cache theo (`fingerprint, model, target`) và chỉ gửi những phát hiện chưa có. Hệ thống chỉ nhận các phân tích có fingerprint khớp với đầu vào, bỏ qua mọi thứ mô hình tự thêm vào. Dòng cache cũ không rõ target được giữ với `target = ''` và không bao giờ được dùng lại. Kiểm tra này nằm trong hàm `get_cached` mà mọi nơi gọi đều phải đi qua.

3.5.5 Tóm tắt, thứ tự ưu tiên và trường hợp bị từ chối

Mỗi lô chỉ thấy phần của nó, nên phần tóm tắt số lượng theo mức độ được tính bằng mã trên toàn bộ phát hiện, không lấy từ lời văn của mô hình. Thứ tự nên xử lý cũng được tính bằng mã: sắp theo mức độ AI đánh giá, rồi theo khả năng false positive (cái chắc chắn là lỗi thật được làm trước). Nếu API trả `stop_reason = "refusal"`, hệ thống ghi rõ là mô hình từ chối, phân biệt với lỗi kỹ thuật, và vẫn xuất báo cáo với phần dữ liệu thô của scanner.

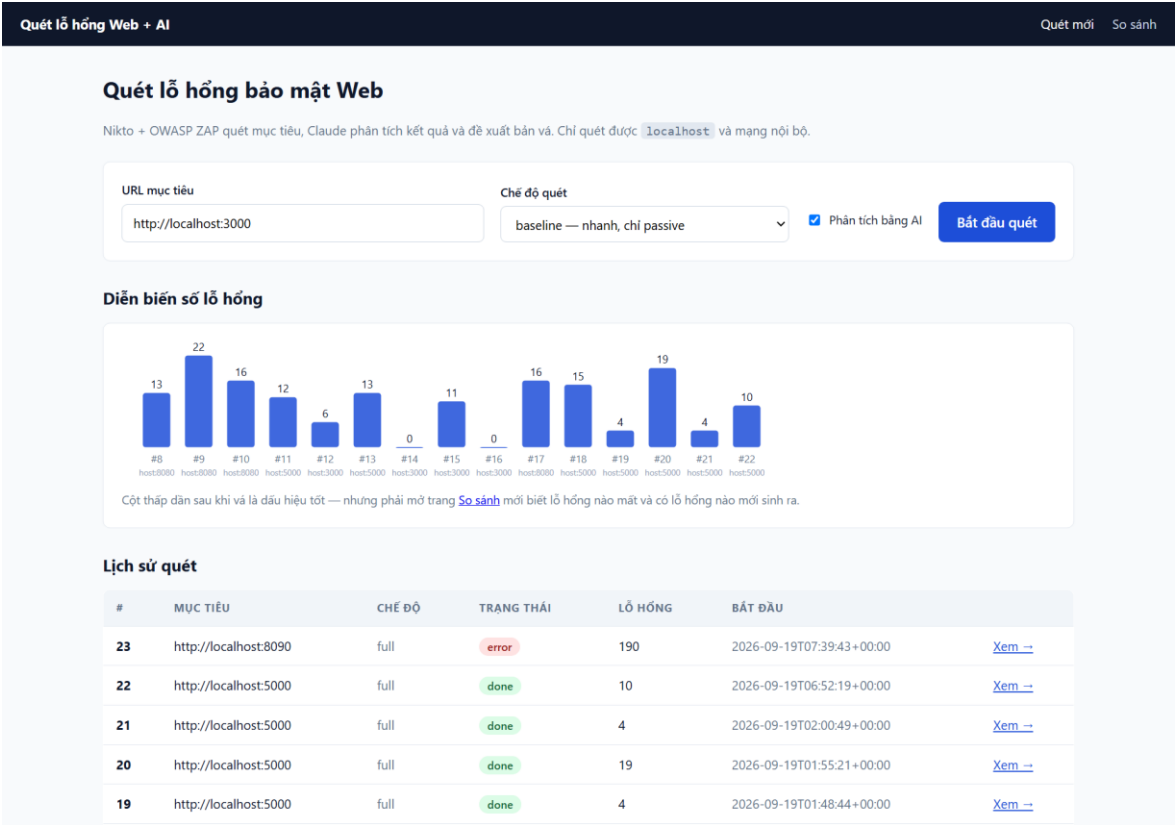
3.6 Backend và giao diện

Bảng 3.5 Các endpoint của dashboard

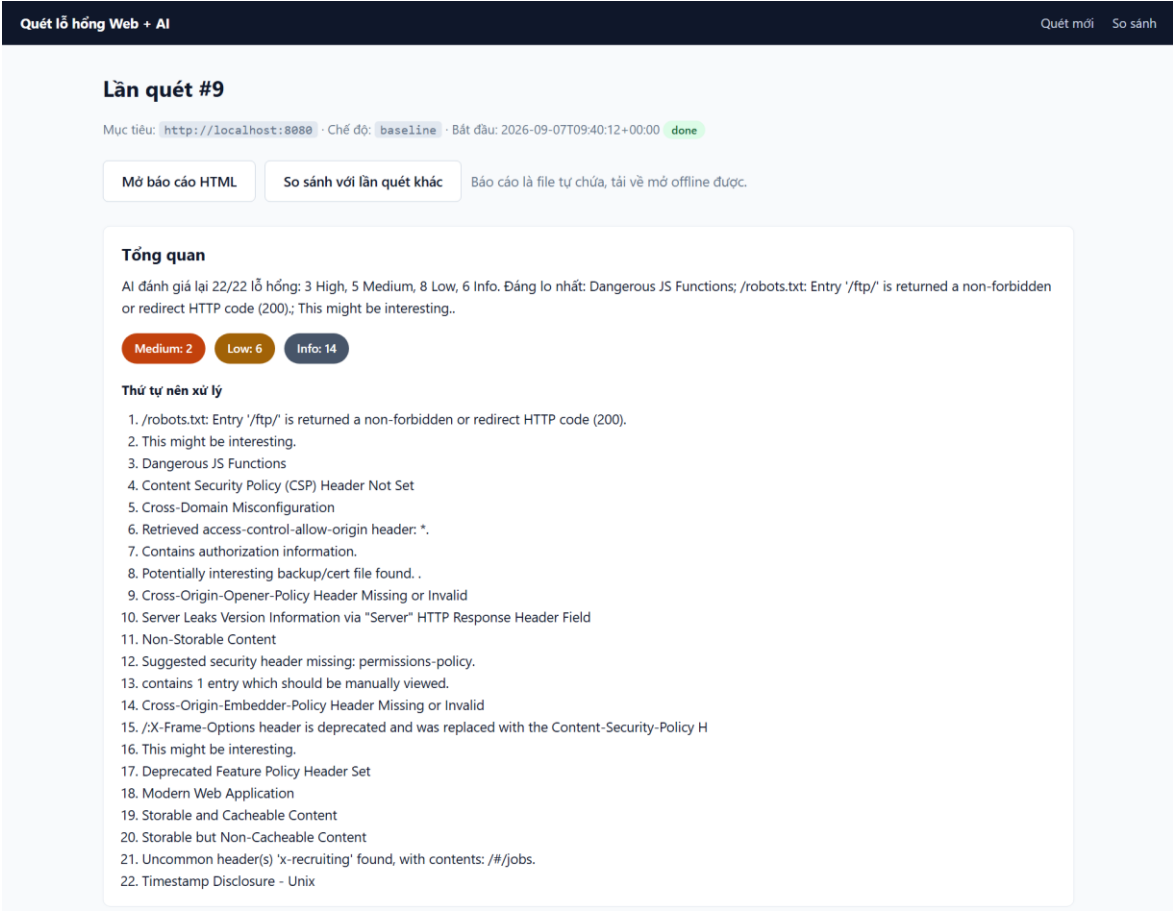
Phương thức, đường dẫn	Chức năng
GET /	Form quét mới, biểu đồ diễn biến số lỗ hổng, lịch sử quét
POST /scans	Kiểm tra phạm vi, tạo lần quét, đẩy vào tác vụ nền, trả mã lần quét ngay
GET /scans/{id}	Chi tiết một lần quét: bảng chứng thô bên trái, phân tích AI bên phải

Phương thức, đường dẫn	Chức năng
GET /scans/{id}/progress	Tiến trình, được HTMX hỏi lại mỗi 2 giây
GET /scans/{id}/report.html	Báo cáo HTML tự chứa, mở offline được
GET /compare?a=&b=	So sánh hai lần quét: đã vá, mới xuất hiện, còn tồn tại

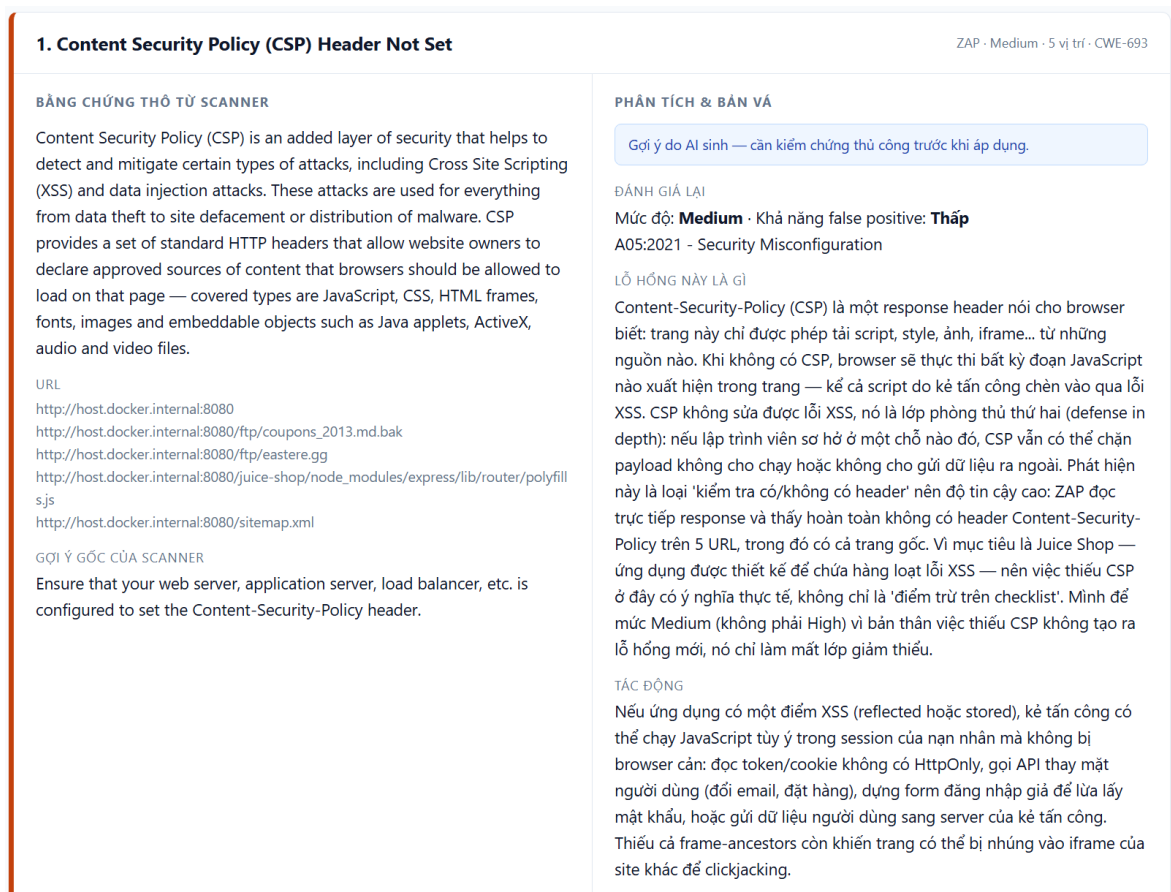
Tác vụ quét chạy bằng `BackgroundTasks` của FastAPI, tiến trình giữ trong một dict trong bộ nhớ. Với ứng dụng chạy cục bộ một tiến trình, hàng đợi như Celery hay Redis là thừa. Giao diện dùng Jinja2 render phía máy chủ cùng HTMX, không cần bước build frontend. HTMX được đóng gói sẵn trong dự án thay vì tải từ CDN, để hôm demo mất mạng vẫn chạy. Mọi phân tích AI trên giao diện đều kèm dòng “Gợi ý do AI sinh — cần kiểm chứng thủ công trước khi áp dụng”.



Hình 3.2 Trang chủ dashboard: form quét, biểu đồ diễn biến số lỗ hổng và lịch sử quét



Hình 3.3 Trang chi tiết lần quét #9: tóm tắt theo mức độ và thứ tự nên xử lý



Hình 3.4 Một phát hiện trên trang chi tiết: bằng chứng thô của scanner (trái) và phân tích, bản vá của AI (phải)

3.7 Các biện pháp an toàn của chính hệ thống

Một công cụ bảo mật mà bản thân nó không an toàn thì phản tác dụng. Các biện pháp sau là phần bắt buộc của thiết kế, không phải tùy chọn:

- **Ranh giới phạm vi quét.** `check_target()` được gọi ở cả CLI lẫn API. Hàm chỉ chấp nhận `http/https` với host là `localhost`, `127.0.0.1`, `:::1`, `host.docker.internal` hoặc địa chỉ IP nội bộ; tên miền bất kỳ bị từ chối. Cờ `--allow-external` tồn tại cho trường hợp có văn bản cho phép, và phải được bật một cách có chủ ý.
- **Không lưu bí mật trong mã nguồn.** Khoá API và tài khoản quét có đăng nhập nằm trong `.env`, file này được loại khỏi git; repo chỉ có `.env.example`.
- **Không tự tạo XSS trong báo cáo.** Bằng chứng và URL là dữ liệu do target kiểm soát. Jinja2 bắt buộc bật `autoescape=True` ở cả báo cáo lẫn dashboard, có test canh việc này.
- **Dashboard không có đăng nhập** và chạy được lệnh `docker`, nên chỉ lắng nghe trên `127.0.0.1` và tuyệt đối không mở ra mạng.

- **Target lab chứa lỗ hổng cố ý chỉ mở trên 127.0.0.1.** vulnapp tự bind 127.0.0.1; các container publish dạng -p 127.0.0.1:<cổng>:<cổng>, scanner trong Docker vẫn tới được qua host.docker.internal. Đã kiểm: gọi Juice Shop từ địa chỉ IP mạng LAN của máy thì không kết nối được. Các lệnh dựng lab ban đầu từng publish ra mọi card mạng; tất cả đã được sửa.
- **Prompt cấm sinh mã khai thác; AI chỉ phục vụ việc hiểu và vá.**

```
def check_target(url: str, allow_external: bool = False) -> str:
    parsed = urlparse(url)
    if parsed.scheme not in ("http", "https"):
        raise TargetNotAllowed(...)
    host = parsed.hostname
    if allow_external or host in _LOCAL_NAMES:
        return url
    try:
        if ipaddress.ip_address(host).is_private:
            return url
    except ValueError:
        pass # là tên miền, không phải IP -> không thuộc mạng nội bộ
    raise TargetNotAllowed(f"Từ chối quét {host!r}: nằm ngoài localhost/mạng nội bộ.")
```

3.8 Kiểm thử phần mềm

Bộ test gồm 59 test chạy hoàn toàn offline, không cần Docker, mạng hay API, trên dữ liệu JSON mẫu của Nikto và ZAP. Các nhóm test chính: đọc và chuẩn hoá kết quả hai scanner; gom trùng và các ca va chạm fingerprint (test hồi quy được kiểm theo kiểu đỏ–xanh: gỡ bản sửa thì test phải hỏng); autoescape của báo cáo; ranh giới phạm vi quét; cache theo target và bước chuyển đổi schema cũ; xuất phiếu gán nhãn (giữ nhãn đã gán, không lộ cột của AI, giữ cột người dùng tự thêm); giới hạn thời gian của full scan.

Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

4.1 Môi trường thực nghiệm

Toàn bộ thực nghiệm chạy trên một máy Windows 11 với Docker Desktop (Docker 29.6) và Python 3.14. Các target được chọn sao cho mỗi cái đóng một vai trò riêng:

Bảng 4.1 Các target trong lab

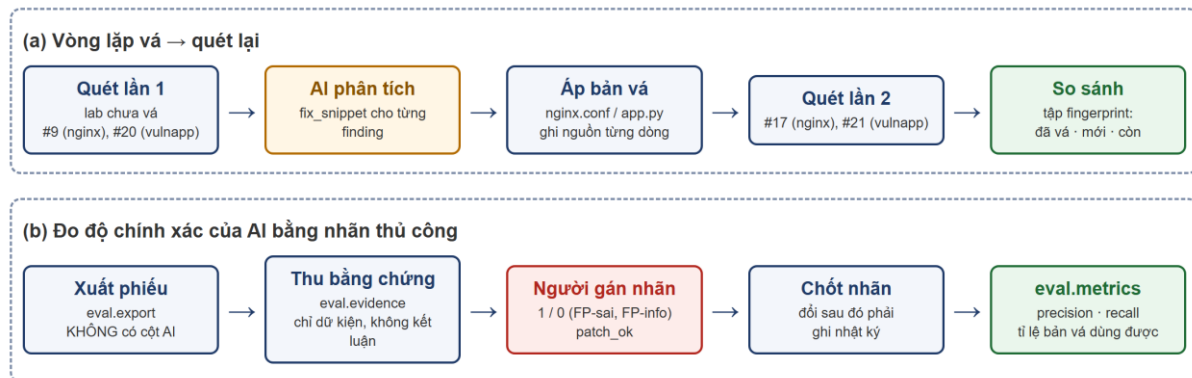
Target	Địa chỉ	Nền tảng	Vai trò
OWASP Juice Shop	localhost:3000	Node.js, Express, Angular	Bề mặt quét phong phú, nhiều lỗ hổng hiện đại; dùng cho quét có đăng nhập
nginx trước Juice Shop	localhost:8080	nginx 1.31.5 (reverse proxy)	Target và được ở tầng cấu hình , vì mã Juice Shop là của người khác
vulnapp	localhost:5000	Flask 3.1.3, SQLite	Ứng dụng khoảng 100 dòng nhóm tự viết, có 4 lỗ hổng cố ý; và ở tầng mã nguồn
OWASP Mutillidae II	localhost:8090	Apache 2.4.7, PHP 5.5.9, kèm phpMyAdmin 3.5.2.2	Target thứ ba, không vá ; mở rộng mẫu đánh giá AI

vulnapp có bốn lỗ hổng cố ý: SQL injection ở /search?q=, lộ câu SQL và lỗi gốc khi truy vấn hổng, XSS phản chiếu ở /greet?name=, và thiếu các security header. Cần cả hai loại target và được: nếu chỉ có nginx thì chỉ chứng minh được AI và cấu hình, không chứng minh được AI và mã.

Bảng 4.2 Các lần quét được dùng trong báo cáo

Lần quét	Target	Profile	Vai trò
#4	Juice Shop :3000	baseline	Đo mức gom trùng trên Juice Shop
#9 → #17	nginx :8080	baseline	Trước vá → sau vá tầng cấu hình
#20 → #21	vulnapp :5000	full	Trước vá → sau vá tầng mã nguồn
#22	vulnapp chưa vá + CSP	full, chỉ ZAP, không AI	Thí nghiệm kiểm tra cảnh báo XSS DOM
#23	Mutillidae :8090	full	Lần đầu; dùng sau 9/32 lô AI vì chi phí (mục 5.11)
#24	Mutillidae :8090	full (AI chạy riêng sau)	Quét lại sau khi sửa khoá gom trùng; dùng để gán nhãn

4.2 Vòng lặp vá → quét lại



Hình 4.1 Hai quy trình đánh giá: (a) vòng lặp vá → quét lại, (b) đo độ chính xác bằng nhãn thủ công

Vòng lặp gồm năm bước: quét bản chưa vá; AI phân tích và sinh `fix_snippet`; áp bản vá; quét lại; so sánh hai tập fingerprint. Để kết quả kiểm chứng được, nhóm đặt các quy tắc sau:

- Bản vá lấy từ `fix_snippet` của AI. Nhóm chỉ được gom các đoạn rời của nhiều phát hiện vào một khối, bỏ phần trùng, và đổi tên cho khớp ứng dụng thật. Mỗi chỗ đổi tên được đánh dấu **CHÍNH**: trong mã.
- Mỗi dòng trong file đã vá ghi rõ tên phát hiện sinh ra nó, để đối chiếu ngược với báo cáo.
- Bản chưa vá được giữ lại (`nginx.conf.chuava.bak`, `app.py.chuava.bak`) để làm lại từ đầu được.
- Với vulnapp, trước khi quét lại, nhóm kiểm tra bằng tay cả việc lỗ hổng đã hết lần việc chức năng còn chạy.

4.3 Xây dựng ground truth

4.3.1 Mẫu

Mẫu gồm 115 cặp (phát hiện, target), gán theo hai đợt. **Đợt 1** gồm 46 cặp từ bốn lần quét của hai target vá được: phát hiện trước vá kiểm trên lab chưa vá, phát hiện chỉ xuất hiện sau vá kiểm trên lab đã vá. **Đợt 2** gồm 72 phát hiện của lần quét #24 trên Mutillidae; 69 gán được, 3 để trống vì không đủ dữ kiện và bị loại khỏi mẫu.

Bảng 4.3 Nguồn của 115 cặp trong mẫu

Target	Lần quét	Số cặp	Thật / FP-sai / FP-info
--------	----------	--------	-------------------------

Target	Lần quét	Số cập	Thật / FP-sai / FP-info
nginx trước Juice Shop, :8080	#9 (22) + #17 (3 mới)	25	5 / 4 / 16
vulnapp Flask, :5000	#20 (19) + #21 (2 mới)	21	11 / 0 / 10
Mutillidae, :8090	#24	69	48 / 7 / 14
Tổng		115	64 / 11 / 40

4.3.2 Quy tắc gán nhãn

- `is_true_positive = 1`: điểm yếu thật trong bối cảnh target.
- `is_true_positive = 0`, chia hai loại. **FP-sai**: scanner khẳng định một điều không đúng. **FP-info**: quan sát đúng nhưng không phải điểm yếu trong bối cảnh này (ví dụ HSTS trên HTTP thuần).
- `patch_ok = 1`: áp đoạn vá của **chính dòng đó** thì phát hiện biến mất và chức năng không hỏng (được đổi tên cho khớp ứng dụng). `patch_ok = 0`: dán nguyên văn thì lỗi, sai tầng, hoặc không có tác dụng. Để trống: không áp thử được.

4.3.3 Kiểm soát thiên lệch

Nguyên cơ lớn nhất của phân đánh giá là gán nhãn theo lời AI, chẳng khác gì chấm bài bằng đáp án của chính người làm bài. Đây cũng là lý do mô hình đang được đánh giá không được tự gán nhãn. Các biện pháp áp dụng:

- **Phiếu mù**: `eval.export` xuất phiếu gán nhãn không có cột nào chứa kết luận của AI.
- **Bảng chứng chỉ gồm dữ kiện**: `eval.evidence` chạy các lệnh kiểm tra (header, CORS, cache, file, robots, OPTIONS...) và chỉ ghi lệnh cùng kết quả thô, không kết luận. Các lỗ hổng cần phán đoán (SQLi, XSS) được người gán tự kiểm chứng bằng curl, DevTools và đọc mã.
- **Chốt nhãn trước khi xem số liệu**. Mọi thay đổi sau đó đều được ghi vào nhật ký kèm lý do và người thực hiện, đồng thời lưu ảnh chụp phiếu ở từng mốc.
- **Không đổi quy tắc sau khi thấy kết quả**. Các phân tích làm thêm sau khi xem số liệu được ghi rõ là phân tích bổ sung, không thay chỉ số chính.
- **Dòng khó được kiểm tay, chỉ bằng chuỗi đánh dấu vô hại**. Ở đợt 2, 9 dòng ban đầu ghi "không thể kiểm chứng" được người gán tự kiểm trên trình duyệt (thay giá trị tham số bằng chuỗi như `zzmark1`, xem mã nguồn và header), chụp màn hình và ghi lại trong file bằng chứng bổ sung.

- **Kiểm tính nhất quán có hệ thống** trước khi người gán xem số liệu: mọi tên cảnh báo xuất hiện ở từ hai target trở lên được đối chiếu nhãn, không chọn dòng theo ảnh hưởng tới số liệu.

4.3.4 Nhật ký chỉnh sửa phiếu

1. Trước khi chạy số liệu, người gán đổi “Cross-Domain Misconfiguration” từ 1 sang 0 (FP-info). Lý do: bước kiểm tra tính nhất quán cho thấy nó và dòng Nikto “Retrieved access-control-allow-origin header” dựa trên cùng một quan sát nhưng mang nhãn ngược nhau.
2. Sau khi xem số liệu `is_true_positive`, người gán xem lại ca XSS DOM và giữ nhãn 1.
3. Theo yêu cầu của người gán, trợ lý AI (Claude) sửa ba ghi chú bị chép nhầm dòng. Nó cũng đổi `patch_ok` của 5 dòng từ 0 sang 1 dựa trên thử riêng từng đoạn vá (người gán đã đánh 0 với lý do “chưa dùng trực tiếp”), và đổi một dòng từ 0 sang trống vì không áp thử được. Mỗi thay đổi đều có ghi chú nguồn gốc; không có nhãn `is_true_positive` nào bị đổi.
4. Người gán xem kết quả thử riêng bản vá XSS DOM (lần quét #22) và đổi `patch_ok` từ 0 sang 1.
5. Đợt 2: phiếu nộp có nhãn nằm nhầm ở cột `patch_ok`; người gán xác nhận, Claude chuyển 72 dòng sang `is_true_positive`. Bản nộp nguyên trạng được lưu lại.
6. Đợt 2: bằng chứng dòng XSS Reflected lần đầu bị sai (dùng token phiên của ZAP trong phiên khác nên phpMyAdmin bỏ tham số). Người gán kiểm lại với phiên hợp lệ và giữ nhãn; ghi chú viết lại theo dữ kiện đúng, kết quả sai vẫn giữ trong file bằng chứng kèm đính chính.
7. Đợt 2: 6/9 dòng khó được gán sau khi có bằng chứng. Người gán đưa nhãn 0/1; loại FP-sai/FP-info quy theo quy tắc chung và được người gán duyệt trước khi ghi. 3 dòng để trống.
8. Đợt 2: bước kiểm tính nhất quán thấy “Modern Web Application” mang 1 ở :8090 nhưng 0 FP-info ở :8080 cho cùng một cảnh báo thông tin; người gán đổi :8090 sang 0 FP-info.

4.4 Chỉ số đánh giá

- **Nhận diện false positive:** lớp dương là “không phải lỗ hổng thật”. AI được tính là dự đoán dương khi và chỉ khi `false_positive_risk = "Cao"`. Định nghĩa này được đặt trong `eval/metrics.py` từ trước khi gán nhãn.
- **Đánh giá lại mức độ:** so mức của AI với mức của scanner, và phân bố mức của AI theo từng loại nhãn.
- **Chất lượng bản vá:** tỉ lệ `patch_ok = 1` trên số dòng có nhãn, tách theo cách kiểm chứng.
- **Giảm khối lượng đọc:** số lần xuất hiện thô so với số phát hiện sau gom.
- **Chi phí:** số token vào/ra và số tiền ước tính.

Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Giảm khối lượng đọc thủ công

Bảng 5.1 Số lần xuất hiện thô và số phát hiện sau gom trùng

Lần quét	Target	Thô	Sau gom	Giảm
#4	Juice Shop : 3000, baseline	196	18	90,8%
#9	nginx : 8080, baseline	200	22	89,0%
#17	nginx : 8080, baseline, sau vá	158	16	89,9%
#20	vulnapp : 5000, full	54	19	64,8%
#21	vulnapp : 5000, full, sau vá	8	4	50,0%
#23	Mutillidae : 8090, full, khoá cũ	371	190	48,8%
#24	Mutillidae : 8090, full, khoá mới	341	72	78,9%

“Thô” là tổng số lần xuất hiện của mọi cảnh báo, sau khi trải các instance của từng cảnh báo. Trên Juice Shop và nginx, gom trùng giảm khoảng 90% khối lượng đọc: người dùng đọc khoảng 20 mục thay vì khoảng 200 dòng. vulnapp nhỏ nên ít trùng lặp hơn.

Mutillidae ở lần quét #23 là ngoại lệ đáng chú ý: sau khi gom, phía Nikto vẫn còn 148 phát hiện. Kiểm tra cho thấy 116/148 phát hiện có cùng một thông báo “Remote File Inclusion (RFI) from RSNAKE's RFI list”, nhưng mỗi mục trong danh sách RFI của Nikto là một test có mã riêng. Khoá gom trùng theo mã test, vốn đúng trên hai target đầu, không gộp được họ phát hiện này. Khoá được sửa để gom họ RFI về một dòng, có test hồi quy; nhưng không gom theo thông báo nói chung, vì ở #9 hai dòng “This might be interesting.” trên /ftp/ và /public/ mang hai nhãn khác nhau.

Lần quét lại #24 không còn dòng RFI nào, nên bản sửa chưa được thử trên dữ liệu thật; lý do RFI biến mất gắn với việc target đã bị hỏng (mục 5.10). Hệ quả về chi phí được trình bày ở mục 5.11.

5.2 Vòng lặp vá trên nginx (tầng cấu hình)

Bảng 5.2 Lần quét #9 (trước vá) và #17 (sau vá) trên nginx

	#9 – trước vá	#17 – sau vá
Nikto, số lần xuất hiện thô	152	117
ZAP, số lần xuất hiện thô	48	41
Tổng thô	200	158
Sau gom trùng	22	16
Banner Server	nginx/1.31.5	nginx

Toàn bộ khối lệnh trong `nginx.conf` được lấy từ `fix_snippet` của lần quét #9; nhóm không tự viết thêm dòng nào. Trích một phần bản vá:

```
# [Server Leaks Version Information via "Server" HTTP Response Header Field]
server_tokens off;
# [Content Security Policy (CSP) Header Not Set]
add_header Content-Security-Policy "default-src 'self'; script-src 'self';
..." always;
# [Cross-Origin-Opener-Policy Header Missing or Invalid]
add_header Cross-Origin-Opener-Policy "same-origin" always;
# [/robots.txt: Entry '/ftp/' is returned a non-forbidden or redirect HTTP
code (200).]
location ^~ /ftp/ { autoindex off; return 404; }
# [Cross-Domain Misconfiguration] - gỡ hẳn CORS của backend
proxy_hide_header Access-Control-Allow-Origin;
```

Quét lỗ hổng Web + AI Quét mới So sánh

So sánh hai lần quét

Chạy quét → áp bản vá AI để xuất → quét lại → so sánh ở đây. Lỗ hổng nằm ở cột **đã vá** là bằng chứng bản vá có tác dụng thật.

A — trước khi vá

#9 — http://localhost:8080 (22 lỗ hổng)

B — sau khi vá

#17 — http://localhost:8080 (16 lỗ hổng)

So sánh

Đã vá 9	Mới xuất hiện 3	Còn tồn tại 13
Info Retrieved access-control-allow-origin header: *. · nikto Info This might be interesting. · nikto Info /robots.txt: Entry '/ftp/' is returned a non-forbidden or redirect HTTP code (200). · nikto Info Non-Storable Content · zap Info Contains authorization information. · nikto	Low Cross-Origin-Resource-Policy Header Missing or Invalid · zap Medium CSP: style-src unsafe-inline · zap Medium CSP: Wildcard Directive · zap	Info Uncommon header(s) 'cross-origin-embedder-policy-report-only' found, with contents: requir · nikto Info /X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy H · nikto Info Potentially interesting backup/cert file found. · nikto Info Storable and Cacheable Content · zap Info Storable but Non-Cacheable Content · zap Info This might be interesting. · nikto Info Suggested security header missing: strict-transport-security. · nikto Info contains 1 entry which should be manually viewed. · nikto Info Modern Web Application · zap
Low Cross-Origin-Opener-Policy Header Missing or Invalid · zap Low Server Leaks Version Information via "Server" HTTP Response Header Field · zap Medium Cross-Domain Misconfiguration · zap Medium Content Security Policy (CSP) Header Not Set · zap		Low Timestamp Disclosure - Unix · zap Low Cross-Origin-Embedder-Policy Header Missing or Invalid · zap Low Deprecated Feature Policy Header Set · zap Low Dangerous JS Functions · zap

So khớp theo fingerprint = sha256(scanner: mã test), nên đối đường dẫn không tạo ra lỗ hổng "mới" giả.

Hình 5.1 Trang so sánh lần quét #9 và #17 trên nginx: 9 đã vá, 3 mới xuất hiện, 13 còn tồn tại

Đã vá 9 phát hiện, gồm “CSP Header Not Set” và “Cross-Domain Misconfiguration” (Medium), COOP và lộ phiên bản qua banner `Server` (Low), cùng 5 phát hiện mức Info (thư mục `/ftp/` trong `robots.txt`, header CORS *, “Contains authorization information”...).

Mới xuất hiện 3 phát hiện, và cả ba đều do chính bản vá gây ra. “CSP: Wildcard Directive”: CSP của AI có `img-src ... https:`, ZAP coi `https:` là wildcard. “CSP: style-src unsafe-inline”: AI tự đưa `'unsafe-inline'` vào `style-src`. “Cross-Origin-Resource-Policy Header Missing”: bật COEP kéo theo yêu cầu CORP, trong khi dòng CORP trong snippet lại bị comment.

Còn tồn tại 13 phát hiện. Một trong số đó che mắt tiến triển thật. Phát hiện Nikto “Suggested security header missing” đi từ 4 lần xuất hiện xuống 1 (chỉ còn HSTS), tức 3/4 header đã được vá. Nhưng lần quét này được ghi trước khi có quy tắc tách theo tên header, nên trang so sánh xếp cả họ vào “còn tồn tại”.

5.2.1 Bản vá của AI sai ở đâu

Bốn lỗi dưới đây được đo bằng `curl -D -` sau khi áp bản vá, không phải nhận định cảm tính:

1. **Header trùng, giá trị mâu thuẫn.** Phản hồi chứa cả `X-Frame-Options: SAMEORIGIN` của Juice Shop lẫn `DENY` của nginx: AI thêm `add_header` nhưng quên `proxy_hide_header`.
2. **Đoạn mã mâu thuẫn với chính lời giải thích.** Phần chữ khuyên bật `Content-Security-Policy-Report-Only` trước, nhưng trong snippet dòng không bị comment lại là bản chặn thật. Hệ quả đo được: CSP chặn luôn script nội tuyến khởi tạo banner cookie của Juice Shop.
3. **Cách xử lý đúng bị để trong comment.** Với “Deprecated Feature Policy Header Set”, AI viết `proxy_hide_header Feature-Policy;` dưới dạng dòng ghi chú, nên header cũ vẫn lộ.
4. **COEP vẫn thiếu, đúng như dự đoán.** Nhóm làm theo khuyến nghị `Report-Only` của AI. Đây là đánh đổi có chủ ý giữa an toàn khi triển khai và việc làm sạch bảng kết quả, không phải bản vá hỏng.

5.3 Vòng lặp vá trên vulnapp (tàng mã nguồn)

vulnapp được quét bằng profile `full` vì ZAP chỉ phát hiện SQL injection khi có active scan. Đoạn mã trước và sau khi vá theo đề xuất của AI:

```
# Trước vá: ghép chuỗi thẳng vào câu truy vấn
sql = f"SELECT username, email, role FROM users WHERE username LIKE '%{q}%'
rows = _conn.execute(sql).fetchall()
```

```
# Sau vá: truy vấn tham số hoá, giới hạn độ dài, không trả lỗi gốc
if len(q) > 100:
    abort(400)
rows = _conn.execute(
    "SELECT username, email, role FROM users WHERE username LIKE ?",
    (f"%{q}%",)
).fetchall()
return render_template("page.html", rows=rows) # Jinja2 tự escape
```

Bảng 5.3 Kiểm chứng bằng tay trên vulnapp trước khi quét lại

Thử	Trước vá	Sau vá
/search?q=' OR '1'='1	Trả về cả 3 user, gồm admin	“Không tìm thấy.”
/search?q=an	Có kết quả	Vẫn có kết quả, chức năng không hỏng
/search?q='	Lộ câu SQL gốc, HTTP 500	HTTP 200, không lộ gì
/greet?name=<script>...	Phản chiếu nguyên vẹn	<script>...
Header Server	Werkzeug/3.1.8 Python/3.14.3	Rỗng

Quét lỗ hổng Web + AI
Quét mới
So sánh

So sánh hai lần quét

Chạy quét → áp bản vá AI đề xuất → quét lại → so sánh ở đây. Lỗ hổng nằm ở cột **đã vá** là bằng chứng bản vá có tác dụng thật.

A — trước khi vá
#20 — http://localhost:5000 (19 lỗ hổng)

B — sau khi vá
#21 — http://localhost:5000 (4 lỗ hổng)

So sánh

Đã vá 17

- High** Cross Site Scripting (Reflected) · zap
- High** Cross Site Scripting (DOM Based) · zap
- High** SQL Injection · zap
- Info** Suggested security header missing: permissions-policy · nikto
- Info** The X-Content-Type-Options header is not set. This could allow the user agent to render th · nikto
- Info** Suggested security header missing: referrer-policy · nikto
- Info** Suggested security header missing: x-content-type-options · nikto
- Info** Storable and Cacheable Content · zap
- Info** Suggested security header missing: content-security-policy · nikto
- Low** Cross-Origin-Embedder-Policy Header Missing or Invalid · zap
- Low** Cross-Origin-Opener-Policy Header Missing or Invalid · zap
- Low** Cross-Origin-Resource-Policy Header Missing or Invalid · zap
- Low** X-Content-Type-Options Header Missing · zap
- Low** Permissions Policy Header Not Set · zap
- Low** Server Leaks Version Information via "Server" HTTP Response Header Field · zap
- Medium** Missing Anti-clickjacking Header · zap
- Medium** Content Security Policy (CSP) Header Not Set · zap

Mới xuất hiện 2

- Info** /X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy H · nikto
- Info** Non-Storable Content · zap

Còn tồn tại 2

- Info** OPTIONS: Allowed HTTP Methods: GET, HEAD · nikto
- Info** Suggested security header missing: strict-transport-security · nikto

So khớp theo `fingerprint` = `sha256(scanner | mã test)`, nên đối đường dẫn không tạo ra lỗ hổng "mới" giả.

Hình 5.2 Trang so sánh lần quét #20 và #21 trên vulnapp: 17 đã vá, 2 mới xuất hiện, 2 còn tồn tại

Đã vá 17/19 phát hiện, gồm cả ba lỗ hổng High: SQL Injection, XSS phản chiều, XSS DOM. Các phát hiện còn lại là CSP, chống clickjacking, COEP/COOP/CORP, Permissions-Policy, X-Content-Type-Options, lộ banner, và các phát hiện tương ứng của Nikto.

Mới xuất hiện 2, cả hai do bản vá gây ra, và AI đánh giá đúng cả hai. Nikto báo “X-Frame-Options header is deprecated” vì bản vá thêm X-Frame-Options: DENY theo yêu cầu của ZAP: hai scanner mâu thuẫn nhau, và AI xếp Low với khả năng false positive Trung bình. ZAP báo “Non-Storable Content” vì Cache-Control: no-store chính là thứ bản vá muốn; AI xếp Info với khả năng false positive Cao.

Còn tồn tại 2. HSTS: AI không vá, và điều đó đúng vì HSTS vô nghĩa trên HTTP thuần; AI tự xếp khả năng false positive Cao. OPTIONS: danh sách phương thức đổi từ HEAD, OPTIONS, GET thành GET, HEAD, nhưng Nikto vẫn liệt kê các phương thức còn lại ở mức Info.

5.3.1 Bản vá của AI sai ở đâu

AI chỉ đọc đầu ra của scanner, không đọc mã nguồn, nên các lỗi tập trung ở chỗ AI phải đoán:

- 1. Dán nguyên văn thì ứng dụng không chạy.** Snippet cho “CSP Header Not Set” có `from flask import ..., escape.flask.escape` đã bị gỡ từ Flask 3.0, nên trên Flask 3.1.3 dòng này báo `ImportError`.
- 2. Đoán tên thay vì nói ra giả định.** Snippet SQLi truy vấn bảng `items(id, name, description)` và `render search.html`; snippet XSS dùng `greet.html`. Không cái nào tồn tại. Nhóm giữ nguyên cách vá và đổi tên cho khớp ứng dụng.
- 3. Năm chuỗi CSP khác nhau cho cùng một ứng dụng.** Sáu phát hiện có đoạn CSP, trả về năm chuỗi khác nhau, mỗi chuỗi trong một hàm `after_request` riêng. Nếu dán hết vào thì các hàm ghi đè lẫn nhau.
- 4. CSP chặn chính CSS của trang.** `style-src 'self'` không cho phép `<style>` nội tuyến, nên trang mất định dạng; chức năng không bị ảnh hưởng.

5.4 Độ chính xác khi nhận diện false positive

Phân bố nhãn trên 115 cặp: 64 lỗi hồng thật, 51 không phải (11 FP-sai, 40 FP-info). Lệnh `python -m eval.metrics all` gộp mọi cặp đã gán nhãn; mỗi cặp chỉ tính một lần, kể cả khi nó có mặt ở nhiều lần quét.

Bảng 5.4 Ma trận nhầm lẫn trên 115 cặp

	AI nói: không phải lỗi hồng	AI nói: lỗi hồng thật
Thật sự không phải lỗi hồng (51)	13	38
Thật sự là lỗi hồng (64)	2	62

Bảng 5.5 Các chỉ số chính, đợt 1 và gộp ba target

Chỉ số	Đợt 1 (46 cặp)	Gộp ba target (115 cặp)
Precision	100% (5/5)	86,7% (13/15)

Chỉ số	Đợt 1 (46 cặp)	Gộp ba target (115 cặp)
Recall	17% (5/30)	25,5% (13/51)
Accuracy	46% (21/46)	65,2% (75/115)

Bảng 5.6 Chỉ số theo từng target

Target	n	Precision	Recall
vulnapp : 5000	21	2/2	2/10
nginx : 8080	25	3/3	3/20
Mutillidae : 8090	69	8/10	8/21

Ở đợt 1, AI không bác nhầm lỗ hổng thật nào, nhưng khi đó chỉ có 5 dự đoán dương, quá ít để khẳng định. Target thứ ba cho thấy tính chất đó không giữ được: AI xếp khả năng false positive “Cao” cho 2 lỗ hổng thật trên Mutillidae (phân tích ở mục 5.7). Precision gộp 86,7% vẫn cao. Kết luận đúng là AI **hiếm khi** bác nhầm lỗ hổng thật, không phải **không bao giờ**. Với một công cụ bảo mật, điều này có nghĩa là không được tự động ẩn các cảnh báo AI xếp “Cao”, chỉ nên xếp chúng xuống cuối danh sách.

5.4.1 Recall thấp đến từ đâu

Bảng 5.7 Khả năng false positive AI đưa ra, theo loại nhãn (gộp)

Nhãn của người	n	AI: Cao	Trung bình	Thấp
FP-sai	11	7	3	1
FP-info	40	6	8	26
Lỗ hổng thật	64	2	13	49

Khi scanner **khẳng định sai**, AI bắt được 7/11. Khi scanner **quan sát đúng nhưng không đáng lo**, AI chỉ gọi 6/40 là false positive. Recall thấp đến chủ yếu từ nhóm thứ hai. Mẫu hình này đã thấy ở đợt 1 (3/4 và 2/26) và lặp lại trên target thứ ba.

Nguyên nhân nằm ở câu hỏi, không nằm ở mô hình. Prompt yêu cầu AI ước lượng false positive theo độ mạnh của bằng chứng, tức là hỏi “scanner có nói sai không”. Còn quy tắc gán nhãn tính cả FP-info là 0, tức là hỏi “có đáng lo không”. Hai trường đang đo hai câu hỏi khác nhau. AI trả lời câu “có đáng lo không” ở một trường khác là **mức độ nghiêm trọng**. Bảng dưới đây là phân tích làm thêm, không thay cho chỉ số chính:

Bảng 5.8 Mức độ AI đánh giá, theo loại nhãn (gộp)

Nhãn của người	Critical	High	Medium	Low	Info
Lỗ hổng thật (64)	3	16	19	23	3

Nhãn của người	Critical	High	Medium	Low	Info
FP-info (40)	0	0	3	17	20
FP-sai (11)	0	2	4	3	2

37/40 dòng FP-info được AI xếp Low hoặc Info, và không dòng FP-info nào lên High. Nói cách khác, AI vẫn phân biệt được cái đáng lo với cái không đáng lo, nhưng thể hiện điều đó qua mức độ chứ không qua trường false positive. Ở đợt 1 không lỗ hổng thật nào bị xếp Info; trên Mutillidae có 3 dòng (Private IP Disclosure, Authentication Request Identified và mục `robots.txt` “contains 8 entries”), đều là quan sát mức thông tin mà người gán tính là điểm yếu thật.

5.4.2 Độ nhạy theo ngưỡng

Nếu tính cả mức “Trung bình” là AI nói false positive thì trên 115 cặp, precision còn 61,5% (24/39) và recall lên 47,1% (24/51). Ở đợt 1, hai lỗ hổng thật bị tính nhầm khi đó là “Dangerous JS Functions” và “Cross Site Scripting (DOM Based)”, cũng là hai ca người gán phải phán đoán nhiều nhất: mức “Trung bình” của AI rơi đúng vào những chỗ khó. Số liệu này chỉ để tham khảo, không thay định nghĩa chính.

5.5 Chất lượng bản vá

`patch_ok` được gán cho 16 lỗ hổng thật. **13/15 bản vá dùng được (87%)**, một dòng để trống.

Bảng 5.9 Tỷ lệ bản vá dùng được theo cách kiểm chứng

Cách kiểm chứng	Dùng được	Không
A. Áp trong vòng lặp vá → quét lại	7	2
B. Áp riêng đoạn vá lên bản chưa vá, chạy thật, đo header và chức năng	6	0
C. Không áp thử được (Dangerous JS Functions: phải sửa mã Angular của Juice Shop)	(trống)	

Bảng 5.10 Tỷ lệ bản vá dùng được theo target

Target	Dùng được
nginx trước Juice Shop, :8080	3/4
vulnapp Flask, :5000	10/11

Cần cách B vì trong vòng lặp, nhóm gom mọi header vào một khối, nên nhiều phát hiện biến mất nhờ đoạn vá của một dòng **khác**. Như vậy vòng lặp không chứng minh được gì cho đoạn vá của chính dòng đó. Với cách B, từng đoạn vá được dán

riêng vào bản chưa vá, chạy thật, kiểm header bằng HTTP và kiểm các chức năng `/search`, `/greet` còn trả đúng.

Hai bản vá không dùng được, mỗi cái một kiểu lỗi. Một là **dán nguyên văn thì ứng dụng không chạy** (CSP trên vulnapp, lỗi `ImportError`). Hai là **sai tầng**: với “This might be interesting” (`/ftp/`) trên `:8080`, đoạn vá chính viết cho Express, còn phương án nginx chỉ nằm trong comment. Phân tích này được sinh trước khi sửa lỗi nhận diện ngăn xếp. Tách theo thời điểm: các phân tích sinh sau khi sửa prompt đạt 11/12; các phân tích sinh trước đạt 2/3.

Một ca được tính là dùng được nhưng cần nói rõ giới hạn là **XSS DOM** trên vulnapp. Phần CSP của đoạn vá làm cảnh báo biến mất và không làm hỏng chức năng, nên đạt quy tắc. Nhưng XSS phản chiếu gốc vẫn còn, vì CSP chỉ chặn việc thực thi trong trình duyệt, và CSP đó còn sinh thêm một cảnh báo mới. Quy tắc `patch_ok` đo “phát hiện có biến mất không”, không đo “lỗi hỏng gốc có được sửa không”. Ca này cho thấy hai điều đó có thể khác nhau.

5.6 AI đánh giá lại mức độ nghiêm trọng

Trên 16 lỗ hổng thật, AI đổi mức ở 9 dòng. Phần lớn là từ Info đi lên, vì Nikto không xếp hạng và để mọi thứ ở Info. Một số thay đổi đáng chú ý:

- Hai dòng về `/ftp/` đi từ Info lên **High**: thư mục liệt kê được cả `coupons_2013.md.bak` và `encrypt.pyc`.
- SQL injection đi từ High lên **Critical**.
- “Dangerous JS Functions” đi từ Low lên **High**.

5.7 Các ca đáng thảo luận

Lời của AI dưới đây được trích nguyên văn từ phân tích lưu trong cơ sở dữ liệu.

CORS Access-Control-Allow-Origin: * trên **Juice Shop**. Người gán đánh 0 (FP-info) cả hai dòng: endpoint thử trả danh mục sản phẩm công khai và không có `Access-Control-Allow-Credentials`, nên trang của origin khác không đọc được phản hồi của request có kèm thông tin xác thực. AI xếp Medium với rủi ro false positive Thấp. AI đồng ý phần credential (“với ACAO: * thì browser KHÔNG gửi cookie/credential kèm theo... nên kẻ tấn công không đọc được dữ liệu riêng tư cần đăng nhập”), nhưng nêu một rủi ro khác: “Nếu ứng dụng chạy trên localhost/mạng nội

bộ, đây là con đường để website bên ngoài dò và trích xuất nội dung dịch vụ nội bộ vốn không expose ra Internet.” Bất đồng thật nằm ở điểm này, vì ứng dụng lab chạy trên localhost.

“**This might be interesting**” trên `/public/`. Người gán đánh FP-sai: URL này trả về cùng mã, cùng kiểu, cùng 9.393 byte với một đường dẫn không tồn tại. Đó là trang `index.html` mà Juice Shop trả cho mọi đường dẫn lạ. AI để rủi ro false positive “Trung bình”, và chính nó đã nêu khả năng này: “Nikto chỉ báo ‘might be interesting’ dựa trên việc response khác 404, nó KHÔNG khẳng định có directory listing... hoặc trả `index.html`”. Đây là dòng FP-sai duy nhất AI không xếp “Cao”.

XSS DOM trên vulnapp. Người gán giữ 1 vì target có XSS thật: payload được phản chiếu nguyên văn ở `/greet`. Nhưng nhãn “DOM-based” thì không khớp với ứng dụng: vulnapp không có dòng JavaScript nào, và payload của ZAP nằm sau dấu #. AI để rủi ro false positive “Trung bình” với lý do: “trường evidence RÕNG... Rất có thể đây là hệ quả lan sang từ Reflected XSS đã xác nhận ở `/search` và `/greet` (ZAP đôi khi báo trùng)”. Thí nghiệm ở lần quét #22 ủng hộ cách hiểu này: chỉ thêm CSP chặn script nội tuyến lên bản chưa vá thì cảnh báo XSS DOM biến mất, còn XSS phản chiếu vẫn còn.

SQL Injection (Time Based) trên phpMyAdmin của Mutillidae. Đây là một trong hai lỗ hổng thật bị AI xếp khả năng false positive “Cao”. Người gán đánh 1: phpMyAdmin vào thẳng bằng root không mật khẩu, và SQL do ZAP gửi đã chạy thật (94 database mang tên payload). AI xếp FP “Cao” nhưng mức độ **High**, với lý do: “phpMyAdmin theo thiết kế nhận tham số `sql_query` và thực thi đúng câu SQL đó — ZAP gửi payload vào đây thì tất nhiên SQL được chạy, nhưng đó là chức năng chứ không phải lỗ hổng injection ... Tuy vậy mình vẫn giữ mức High, không phải vì tin cảnh báo injection, mà vì bằng chứng này phơi bày một sự thật nghiêm trọng hơn: phpMyAdmin đang mở công khai”. Người và AI thống nhất về bản chất. Chỗ khác nhau là trường FP của AI trả lời “ZAP gọi tên có đúng không”, còn nhãn trả lời “có điểm yếu thật không”. Đây cùng hiện tượng với ca XSS DOM ở trên.

Information Disclosure – Suspicious Comments. Lỗ hổng thật thứ hai bị AI xếp FP “Cao”, và là bất đồng thật. Người gán đánh 1 (“lộ comment nội bộ của lập trình viên”). AI xếp mức Low, cho rằng đây là kiểm tra theo từ khoá và “nhiều khả

năng là một comment mang tính giảng dạy trong Mutillidae”. Comment bị bắt nằm trong file tài liệu `documentation/Mutillidae-Test-Scripts.txt`.

5.8 Quét có đăng nhập trên Juice Shop

Juice Shop là SPA viết bằng Angular, nên spider truyền thống gần như chỉ thấy `index.html`. Do thật: spider truyền thống tìm được 149 URL, AJAX Spider trong profile `auth` tìm được 1.077 URL. Profile `auth` tìm thêm 7 phát hiện mà `baseline` bỏ sót, gồm “Session ID in URL Rewrite” (Medium) và “Application Error Disclosure”. Hai cảnh báo “Authentication Request Identified” và “Session Management Response Identified” là bằng chứng ZAP đã thật sự nhận ra luồng đăng nhập.

5.9 Lỗi của chính công cụ do vòng lặp và target thứ ba phát hiện

Các lỗi dưới đây không lộ ra qua test đơn vị. Chúng chỉ xuất hiện khi chạy vòng lặp thật trên nhiều target, và là một kết quả đáng giá của đồ án.

- **Cache dùng chung giữa các target.** Khoá cache ban đầu chỉ có fingerprint. Lần quét #18 của vulnapp lấy 7 phân tích từ cache của target nginx, và báo cáo khuyên `server_tokens off` cho phiên bản nginx/1.31.5, trong khi đó là một ứng dụng Flask không có nginx nào. Khoá được sửa thành `(fingerprint, model, target)`. 33 dòng cache cũ được gán lại target theo lịch sử đã đối chiếu. Quy tắc đoán tự động “lần quét gần nhất có fingerprint đó” cũng đã được thử, nhưng gán sai 18/33 dòng nên không dùng.
- **Nikto dùng chung một mã cho cả họ phát hiện.** Header lạ `x-recruiting` đã được vá nhưng bị gộp với một header lạ mới xuất hiện, nên trang so sánh báo “còn tồn tại”. Năm header bảo mật bị thiếu bị gộp làm một, và phát hiện về HSTS hiển thị bản vá AI viết cho CSP. Lỗi này được sửa bằng quy tắc khoá ở mục 3.3.3.
- **Bản vá xoá mất tín hiệu mà bộ phân tích cần.** `server_tokens off` và banner rỗng làm `detect_stack()` trả về rỗng ở các lần quét sau vá: vá càng kỹ thì lần quét sau càng khó nhận diện ngăn xếp. Cache theo target che bớt vấn đề này, nhưng 3 phát hiện mới ở #17 đã được phân tích khi không có thông tin ngăn xếp.
- **Khoá gom trùng Nikto chưa đủ cho mọi target** (Mutillidae, mục 5.1): 116 phát hiện RFI giống hệt nhau từng thành 116 dòng riêng. Đã sửa kèm test hồi quy; chưa thử lại được trên lần quét thật.

Target thứ ba còn làm lộ ba lỗi nhỏ hơn, đều đã sửa:

- Kiểm tra SQLi/XSS trong script thu bằng chứng viết riêng cho vulnapp (/search?q=, tham số name). Chạy trên Mutillidae thì chúng ghi dữ kiện của một trang không liên quan, dễ khiến người gán đánh nhầm. Giờ chúng chỉ chạy đúng trên vulnapp; target khác chỉ gọi lại URL scanner báo, kèm ghi chú “tự kiểm chứng”.
- File .git/index lộ trên Mutillidae đưa ký tự nhị phân vào file bằng chứng dạng markdown. Giờ các ký tự điều khiển được thay bằng dấu chấm giữa.
- Một lô phân tích trả 0/6 kết quả mà log vẫn ghi [ok]. Log giờ đánh dấu [!] và cho biết mô hình trả thiếu hay trả sai fingerprint; chạy lại lô đó được đủ 6/6.

5.10 Target bị chính active scan làm hỏng

Trên Mutillidae, active scan của ZAP đã làm thay đổi chính target. Các dữ kiện đo được:

- phpMyAdmin 3.5.2.2 đi kèm image tự đăng nhập MySQL bằng root **không mật khẩu**. Trang của nó tự cảnh báo điều này. Người gán mở trong cửa sổ ẩn danh và vào thẳng trang quản trị.
- ZAP dò được phpMyAdmin rồi chạy các payload của nó qua đó. Kết quả là MySQL có 94 database, phần lớn mang tên payload của ZAP ("<script>alert(1);</script>, ../../etc/passwd...).
- Lúc quét #24 và lúc gán nhãn, mọi trang của Mutillidae chuyển sang database-offline.php, báo “Access denied for user 'admin'”. Tài khoản MySQL admin mà ứng dụng dùng không còn. Đây là tài khoản gốc của image (xem đoạn dưới); nó mất sau khi container khởi động, nguyên nhân chưa xác định.
- Lần quét #23 có 116 dòng RFI, #24 không có dòng nào. Giả thuyết chưa kiểm chứng là index.php giờ chỉ còn chuyển hướng nên các test RFI không khớp.

Sau khi chốt nhãn, nhóm dựng lại container từ image gốc và khởi tạo lại cơ sở dữ liệu: trang chủ trả 200, MySQL còn 4 database. Việc dựng lại cũng trả lời một câu hỏi còn treo. Trong container mới, file cấu hình MySQLHandler.php được ghi lại 6 giây sau khi container khởi động, và trang cài đặt kết nối MySQL bằng tài khoản admin. Tức là chính script khởi động của image tạo tài khoản và ghi file này. Mốc sửa file ở

container cũ (49 giây trước lần quét #23) chỉ là lúc container khởi động, không phải một lần can thiệp tay.

Nhân của đợt 2 được gán theo đúng trạng thái lúc quét #24, nên phép đo vẫn nhất quán. Nhưng đây là một Mutillidae đã hỏng, không phải bản nguyên gốc. Bài học rút ra: active scan có thể sửa, xoá dữ liệu của target, nhất là khi gặp một công cụ quản trị mở không cần đăng nhập. Đây là lý do thực tế cho ranh giới “chỉ quét hệ thống của mình”, và cho việc chụp lại (snapshot) target trước mỗi lần quét chủ động để khôi phục được.

5.11 Chi phí vận hành

Chi phí được ước tính từ số token do API trả về, theo đơn giá công bố của mô hình tại thời điểm thực hiện:

Bảng 5.11 Chi phí API đo được

Trường hợp	Token vào / ra	Chi phí ước tính
#9: nginx, 22 phát hiện, lần đầu	18.437 / 47.293	≈ 1,27 USD (khoảng 33.500đ), tức khoảng 1.500đ mỗi phát hiện
#17: quét lại nginx sau vá	Chỉ gọi API cho 3/16 phát hiện	13/16 lấy từ cache
#23: Mutillidae, khoá cũ, dừng sau 9/32 lô (54/190 phát hiện)	48.252 / 105.916	≈ 2,9 USD; chạy hết dự kiến khoảng 10 USD
#24: Mutillidae, khoá mới, 72 phát hiện (46 lấy từ cache, 26 gọi mới, 6 chạy lại)	27.106 / 51.663	≈ 1,4 USD

Hai nhận xét rút ra. Thứ nhất, token **đầu ra** chiếm phần lớn chi phí, vì mỗi phát hiện đi kèm giải thích và đoạn mã vá. Thứ hai, chi phí tỉ lệ thuận với số fingerprint, nên **chất lượng gom trùng quyết định trực tiếp chi phí**. Ở #23, khoảng 116 phát hiện RFI gần như giống hệt nhau đã được gửi đi phân tích riêng lẻ. Nhóm dừng lần quét sau 9 lô để giữ ngân sách, sửa khoá gom trùng, rồi quét lại (#24, không gọi AI) và chỉ phân tích phần còn thiếu. 46 trong 54 phân tích đã trả tiền ở #23 được dùng lại qua cache; 6 dòng RFI mang fingerprint cũ và 2 dòng không còn xuất hiện ở #24 thì không. Tổng cho Mutillidae khoảng 4,3 USD thay vì khoảng 10 USD.

Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

- Xây dựng hoàn chỉnh hệ thống nối Nikto và OWASP ZAP (ba profile baseline, full, auth) với mô hình Claude, có CLI, dashboard, lịch sử, so sánh hai lần quét và báo cáo HTML tự chứa; khoảng 3.200 dòng Python và 59 test offline.
- Gom trùng giảm khoảng 90% khối lượng đọc trên Juice Shop và nginx, và 79% trên Mutillidae sau khi sửa khoá cho họ RFI.
- Vòng lặp vá → quét lại thành công trên cả hai tầng: nginx 22 → 16 (9 đã vá), vulnapp 19 → 4 (17 đã vá, gồm SQL injection và hai lỗ hổng XSS mức High).
- Đo được độ chính xác của AI trên 115 cặp có nhãn của ba target: precision 86,7% (13/15), recall 25,5% (13/51); 87% bản vá dùng được (13/15).
- Chỉ ra được nguyên nhân của recall thấp (trường đo và câu hỏi gán nhãn không khớp nhau). Mẫu hình này lặp lại trên target thứ ba: AI bắt 7/11 dòng scanner khẳng định sai nhưng chỉ 6/40 dòng quan sát đúng mà không đáng lo, trong khi xếp 37/40 dòng đó ở Low/Info.

6.2 Bài học rút ra

- **Có đo mới biết.** Bốn lỗi của công cụ chỉ lộ ra khi chạy vòng lặp thật trên nhiều target; test đơn vị đều qua.
- **Câu hỏi đánh giá phải khớp với trường được đo.** Hỏi “scanner có sai không” và “có đáng lo không” là hai câu khác nhau, và cho ra hai con số rất khác nhau.
- **AI không thấy mã nguồn thì sẽ đoán tên.** Cách vá thường đúng, nhưng chi tiết (tên bảng, tên template, API đã bị gỡ) cần người kiểm lại.
- **Bản vá có thể sinh cảnh báo mới.** Cả hai vòng lặp đều có phát hiện mới do chính bản vá gây ra, nên sau khi vá luôn phải quét lại.
- **Chất lượng gom trùng là chi phí.** Một khoá gom trùng không phù hợp với target mới có thể làm chi phí tăng gấp nhiều lần.
- **Active scan có thể phá hỏng target.** Trên Mutillidae, ZAP chạy SQL thật qua một phpMyAdmin mở không mật khẩu. Cần chụp lại target trước mỗi lần quét chủ động.

- **Kết luận trên một mẫu nhỏ dễ bị lật.** “AI không bác nhầm lỗ hổng thật nào” đúng trên 46 cặp nhưng không còn đúng khi thêm target thứ ba.

6.3 Hạn chế

- Chưa đo được mức đồng thuận giữa người gán: đợt 1 một người gán, đợt 2 hai người chia nhau các dòng, không gán trùng.
- Mẫu nhỏ: 115 cặp, trong đó 11 FP-sai và 15 dự đoán dương.
- Mutillidae bị hỏng trong lúc quét (mục 5.10); 3 dòng của nó để trống; nhãn nghiêng về lỗ hổng thật (48/69) vì đây là ứng dụng cố ý có lỗ hổng.
- `patch_ok` của 6 dòng dựa trên thử nghiệm tự động do trợ lý AI chạy. Kết quả là đủ kiện đo được (header có mặt, chức năng còn chạy), nhưng người thực hiện không phải người gán.
- Ranh giới giữa FP-info và 1 ở nhóm “thiếu header” phụ thuộc quy tắc người gán tự chốt.
- Cả ba target trong mẫu đánh giá đều do nhóm dựng hoặc chọn; vulnapp và Mutillidae cố ý có lỗ hổng. Chưa có lần full scan nào trên Juice Shop. Bản sửa khoá gom trùng RFI chưa được thử trên lần quét thật.
- Phân tích được cache theo fingerprint. Khi chi tiết cảnh báo đổi mà fingerprint giữ nguyên, báo cáo vẫn hiện lời giải thích của lần đầu.
- Mọi bản vá AI đề xuất đều phải được kiểm chứng thủ công trước khi áp dụng; hệ thống chưa tự kiểm chứng bản vá.

6.4 Hướng phát triển

- Tách hai câu hỏi trong schema và trong nhãn: “scanner có đúng không” và “có cần xử lý không”, rồi đo mỗi trường AI với đúng câu hỏi của nó.
- Cho hai người gán **cùng** một tập dòng để đo độ đồng thuận. Container Mutillidae đã được dựng lại sạch; bước tiếp theo là quét lại nó để có mẫu không bị hỏng, lần này chụp lại trạng thái trước khi quét chủ động.
- Hiện thị ước tính chi phí trước khi gọi API, kèm giới hạn ngân sách cho mỗi lần quét; thử lại khoá gom trùng RFI khi họ này xuất hiện lại.
- Cho AI đọc thêm mã nguồn (khi có) để tránh đoán tên, và cho phép ghi đè ngăn xếp theo target khi banner đã bị ẩn.

- Tự động kiểm chứng bản vá: áp từng đoạn vá trong container riêng, chạy lại đúng luật quét liên quan và kiểm tra chức năng.

TÀI LIỆU THAM KHẢO

- [1] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] MITRE, “Common Weakness Enumeration (CWE).” [Online]. Available: <https://cwe.mitre.org/>
- [3] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical Guide to Information Security Testing and Assessment,” NIST Special Publication 800-115, 2008.
- [4] A. Doupé, M. Cova, and G. Vigna, “Why Johnny Can't Pentest: An Analysis of Black-Box Web Vulnerability Scanners,” in Proc. DIMVA, 2010.
- [5] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, “State of the Art: Automated Black-Box Web Application Vulnerability Testing,” in Proc. IEEE Symposium on Security and Privacy, 2010.
- [6] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, “Examining Zero-Shot Vulnerability Repair with Large Language Models,” in Proc. IEEE Symposium on Security and Privacy, 2023.
- [7] G. Deng et al., “PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing,” in Proc. USENIX Security Symposium, 2024.
- [8] M. Fu, C. Tantithamthavorn, V. Nguyen, and T. Le, “ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?,” in Proc. Asia-Pacific Software Engineering Conference (APSEC), 2023.
- [9] ZAP Project, “ZAP Documentation” và “ZAP Docker Packaged Scans.” [Online]. Available: <https://www.zaproxy.org/docs/>
- [10] C. Sullo, “Nikto web server scanner.” [Online]. Available: <https://github.com/sullo/nikto>
- [11] OWASP Foundation, “OWASP Juice Shop.” [Online]. Available: <https://owasp.org/www-project-juice-shop/>
- [12] OWASP Foundation, “OWASP Secure Headers Project.” [Online]. Available: <https://owasp.org/www-project-secure-headers/>
- [13] MDN Web Docs, “Content Security Policy (CSP).” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

- [14] MDN Web Docs, “Cross-Origin Resource Sharing (CORS).” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- [15] Anthropic, “Claude API Documentation.” [Online]. Available: <https://docs.anthropic.com/>
- [16] J. Druin, “OWASP Mutillidae II.” [Online]. Available: <https://github.com/webpwnized/mutillidae>
- [17] Quốc hội nước CHXHCN Việt Nam, Luật An ninh mạng, số 24/2018/QH14, 2018.
- [18] Quốc hội nước CHXHCN Việt Nam, Bộ luật Hình sự, số 100/2015/QH13, sửa đổi, bổ sung bởi Luật số 12/2017/QH14.
- [19] S. Ramírez, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [20] nginx, “Module ngx_http_headers_module.” [Online]. Available: https://nginx.org/en/docs/http/ngx_http_headers_module.html
- [21] OWASP Foundation, “OWASP Cheat Sheet Series: SQL Injection Prevention; Cross Site Scripting Prevention.” [Online]. Available: <https://cheatsheetseries.owasp.org/>

PHỤ LỤC: HƯỚNG DẪN CHẠY LẠI

Yêu cầu: Docker Desktop đang chạy, Python 3.12 trở lên, một Anthropic API key trong file `.env`.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env          # điền ANTHROPIC_API_KEY
docker pull ghcr.io/sullo/nikto
docker pull ghcr.io/zaproxy/zaproxy:stable
docker run --rm -d -p 127.0.0.1:3000:3000 --name juiceshop bkimminich/juice-shop
docker run -d --name mutillidae -p 127.0.0.1:8090:80 citizenstig/nowasp
curl.exe -s http://localhost:8090/set-up-database.php > NUL # khởi tạo DB Mutillidae
# nginx và vulnapp: xem lab/README.md

python scan.py http://localhost:3000 --no-ai          # chỉ scanner, không tốn tiền API
python scan.py http://localhost:3000 --open          # quét -> AI -> báo cáo HTML
python scan.py --compare 9 17                        # so sánh trước/sau khi vá
uvicorn api.main:app                                  # dashboard:
http://127.0.0.1:8000

python -m eval.export 9                               # xuất phiếu gán nhãn (không có cột AI)
python -m eval.evidence 9                             # thu bằng chứng dạng dữ kiện
python -m eval.metrics 9                             # precision, recall, tỉ lệ bản vá dùng được của một lần quét
python -m eval.metrics all                            # gộp mọi cặp đã gán nhãn trên mọi target
pytest -q                                             # 59 test offline
```

Quy trình chi tiết của hai vòng lặp nằm trong `lab/KETQUA_VONG_LAP.md`; quy trình gán nhãn nằm trong `eval/HUONG_DAN_GAN_NHAN.md`; toàn bộ số liệu đánh giá nằm trong `eval/KETQUA_DANH_GIA.md`.